

Fair Allocation with Optional Selling

Uriel Feige and Yotam Gafni

Weizmann Institute of Science
{uriel.feige, yotam.gafni}@weizmann.ac.il

Abstract. We consider fair allocation of indivisible goods in a setting in which agents have subjective valuation functions over the set of goods, and in addition, goods may be sold at given market prices. In this setting, a fair allocation involves deciding which goods to sell, how to allocate the unsold goods, and how to divide the money received from the sold goods. We adapt to this setting the definitions of share-based fairness notions, such as the maximin share (MMS) and the truncated proportional share (TPS), and comparison-based fairness notions such as EF1 and EFX (which we adapt to SEF1 and SEFX). We show the following results when the utility of each agent is additive both over goods and over money. With two agents, there are allocations that are simultaneously MMS and SEFX. With three agents, there are instances in which no allocation gives every agent more than $\frac{11}{12}$ -MMS. With any number of agents, there are $\frac{2}{3}$ -MMS allocations. There also are allocations that are simultaneously SEFX and $\frac{n}{2n-1}$ -TPS. This latter ratio is best possible, even without the SEFX requirement.

1 Introduction and Preliminaries

The problem of Fair Division, popularized by the Cut & Choose cake-cutting procedure [36] has seen immense theoretical attention and progress in the last decade, with applications in diverse fields such as rent and land division [24,35], course allocation [15], and household chores management [29]. However, one assumption that is absent in the core-model of indivisible items is that of an outside option, in the form of a market price. Consider, for example, a couple going through divorce with a single shared property. If we consider the house as an indivisible good, then there is no fair settlement: Only one of the partners can have it. However, a natural solution is to sell the house and split the proceeds equally. In formal fair allocation terms, we could say that in the purely indivisible setting, the Maximin Share (MMS) of each agent is 0, but with selling, their maximin share is half of the house’s market-price. Since selling only opens up more options, a definition of MMS for this setting always (weakly) outperforms the MMS guarantee in the purely indivisible setting. As we show, this superior guarantee is provably harder to approximate in some settings than the usual MMS without selling (see Table 1). Generally, a useful distinction in fair allocation problems is between *share*-based and *envy*-based notions, where the former are minimal utility guarantees each agent can expect, while the latter is a comparison-based guarantee for how agents view other’s bundles in the final outcome. Since in the setting we introduce with selling, there is always the option of selling all items and sharing the proceeds equally, achieving an envy-free outcome is straight-forward. However, when the subjective valuations far supersedes the market price, this results in arbitrarily bad share guarantees. Thus, many of our results aim at a “Best of Both Worlds” guarantee: Optimal share approximation, together with a strong envy-free notion, adapted to our setting (SEFX).

1.1 Our Fairness Notions

In this section we introduce and motivate the fairness notions that we consider. They are adaptations of fairness notions that are commonly used in settings without sellable goods. The main share-based fairness notion that we will consider is the *maximin share* (MMS), and we shall also consider the

truncated proportional share (TPS). For envy-based fairness notions, we consider SEFX and SEF1, which are relaxations of envy-freeness adapted to our setting.

We consider a fair allocation problem with a set $\mathcal{N} = \{1, 2, \dots, n\}$ of agents and a set $\mathcal{M} = \{g_1, g_2, \dots, g_m\}$ of indivisible goods. We may use the term “item” instead of “good”. Each agent i has a subjective valuation $v_i : 2^{\mathcal{M}} \rightarrow \mathbb{R}_{\geq 0}$. We assume that valuations are additive ($v(S) = \sum_{g \in S} v(g)$) and non-negative. In addition, there are market prices for the goods $p : \mathcal{M} \rightarrow \mathbb{R}_{\geq 0}$. Let $\mathcal{S} \subseteq \mathcal{M}$ be the subset of goods which are sold in a certain outcome. Given the sold goods $\mathcal{S}$, we get $\sum_{g \in \mathcal{S}} p(g)$ in sale proceeds, and are left with the unsold goods. We assume the sale proceeds are in the form of money, meaning they are divisible and have the same (additive) value for all agents. We can then decide a partition $((A_1, P_1), \dots, (A_n, P_n))_{\mathcal{S}}$ that allocates the unsold goods and the money from the sale. For feasibility, the following condition must hold w.r.t. $(\mathcal{S}, A, P)$:

- All allocated kept goods are not sold:

$$\mathcal{S} \cap (\cup_{i=1}^n A_i) = \emptyset, \quad (1)$$

- Distributed sale proceeds do not exceed the proceeds from sold goods:

$$\sum_{i=1}^n P_i \leq \sum_{g_j \in \mathcal{S}} p(g_j). \quad (2)$$

Moreover, if the allocation is a *full* allocation (rather than a *partial* allocation), it must hold that all goods are either allocated or sold, and the distributed sale proceeds equal the proceeds from sold goods:

$$\mathcal{S} \cup \bigcup_{i=1}^n A_i = \mathcal{M}, \quad \sum_{i=1}^n P_i = \sum_{g_j \in \mathcal{S}} p(g_j). \quad (3)$$

We assume the agents utilities are *additive* w.r.t. both their subjective valuations and the sale proceeds, i.e., given a bundle B_j that consists of allocated kept goods A_j and sale proceeds P_j , the agent utility is:

$$u_i(A_j, P_j) = \sum_{g \in A_j} v_i(g) + P_j. \quad (4)$$

Definition 1. *Proportional Share (PS).*

$$PS(\mathcal{M}, v, p, n) = \frac{1}{n} \sum_{j \in \mathcal{M}} \max\{p(j), v(j)\}.$$

Even in the setting without selling, the proportional share is not attainable or approximable (consider one item and two agents with low market price and very high subjective valuations).

Definition 2. *For agent $i \in \mathcal{N}$, the maximin share (MMS) is the maximum value that agent i can guarantee for themselves if they were to optimally sell the goods, and then partition the remaining goods and sale proceeds into n bundles, where the agent receives the least valuable bundle according to their own valuation. Formally:*

$$MMS_i(\mathcal{M}, v, p, n) = \max_{(\mathcal{S}, A, P)} \min_{j \in \{1, 2, \dots, n\}} u_i(A_j, P_j), \quad (5)$$

where the maximum is taken over all valid outcomes $(\mathcal{S}, A, P)$. When the parameters $\mathcal{M}, v, p, n$ are clear from context, we use MMS_i in short. We say that an outcome is ρ -MMS for some $0 \leq \rho \leq 1$ if each agent i gets a utility of at least $\rho \cdot MMS_i$ in that outcome.

Observe that unlike the definition of MMS in settings without sellable goods, in Definition 2 the agent gets to optimize two aspects: both the decision of which goods to sell, and the decision of how to do the partition for the remaining goods (and sale proceeds). For the purpose of determining the MMS value, it will always benefit an agent to sell those goods e for which $p(e) \geq v_i(e)$. Moreover, it may also benefit the agent to sell some goods e with $v_i(e) > p(e)$. For example, this is necessary in order to get positive MMS value whenever $m < n$.

Truncated Proportional Share (TPS) was previously introduced for the case of indivisible goods without selling [7]. We suggest the following extension to our case:

Definition 3. *Truncated Proportional Share With Selling (from here onwards, TPS): With a set of agents $\mathcal{N}$, set of goods $\mathcal{M}$, a valuation function v and a market-price p , define $TPS(\mathcal{M}, v, p, n)$ as the largest value t so that*

$$\frac{1}{n} \sum_{j \in \mathcal{M}} \max\{p(j), \min\{v(j), t\}\} = t.$$

We note that if $p = 0$, the definition takes the same form as TPS for the setting without selling. We also show:

Lemma 1. $MMS(\mathcal{M}, v, p, n) \leq TPS(\mathcal{M}, v, p, n) \leq PS(\mathcal{M}, v, p, n)$.

Before defining envy-based fairness notions, we introduce some notation. Given an additive valuation function v_i and item price function p , we define a modified valuation function $\bar{v}_i$ as follows. For every good $g \in \mathcal{M}$, $\bar{v}_i(g) = \max\{p(g), v_i(g)\}$, and for every set $S \subset \mathcal{M}$, $\bar{v}_i(S) = \sum_{g \in S} \bar{v}_i(g)$. The use of $\bar{v}_i$ is natural when we give the agent autonomy to decide if the unsold goods in their possession are kept or sold.

Definition 4. *An allocation $(A_1, P_1), \dots, (A_n, P_n)$ is envy free (EF) if for every agent i and other agent j , $\bar{v}_i(A_i) + P_i \geq \bar{v}_i(A_j) + P_j$. An allocation satisfies the following relaxations of EF if the above envy free condition holds whenever $P_j > 0$, and either the EF condition or the following conditions hold if $P_j = 0$.*

1. *SEF1 (strong/sellable envy free up to one good). $\bar{v}_i(A_i) + P_i \geq \bar{v}_i(A_j \setminus \{g\}) + p(g)$ for some good $g \in A_j$.*
2. *SEFX (strong/sellable envy free up to any good). $\bar{v}_i(A_i) + P_i \geq \bar{v}_i(A_j \setminus \{g\}) + p(g)$ for every good $g \in A_j$.*

Definition 4 differs in several aspects from the standard definitions of envy-based fairness that are used when there are no sellable goods. To motivate these aspects, consider the simple setting of $n = 2$ and $m = 1$. Suppose that the single good g has positive market price $p(g) > 0$. Then, if we wish the outcome to qualify as satisfying envy-based fairness notions, there seems to be only one acceptable outcome, which is to sell g and give each agent $\frac{p(g)}{2}$. Consequently, we wish EF and any relaxation of EF (such as SEF1 and SEFX) to enforce this outcome, while a straight-forward application of EFX from the setting without selling would also accept allocating the good in full to one of the agents. Definition 4 indeed does enforce this outcome (not just for EF, but also for SEFX and SEF1).

1.2 Our Results

All results in this section are stated for additive valuations in the setting of indivisible goods with optional selling.

We first present combined share & envy notions results, where we combine tight share approximation guarantees with our strongest approximation notion of SEFX:

Theorem 1. *For two agents with additive valuations, there is an allocation that is simultaneously SEFX and MMS.*

In the setting without sellable items, a theorem analogous to Theorem 1 is proved via the Cut & Choose allocation algorithm. In Appendix 3.1 we show that with sellable items, Cut & Choose guarantees only $\frac{1}{2}$ -MMS. To prove Theorem 1, we introduce a *Cut & Give* paradigm, which is a variation on Cut & Choose. See more details in Section 3.1.

The other share notion we consider is the Truncated Proportional Share (TPS) [7]. In settings without sellable items, it was shown in [7] that $\frac{n}{2n-1}$ -TPS allocations always exist, and that $\frac{n}{2n-1}$ is best possible. Our main theorem concerning the TPS is the following.

Theorem 2. *Every allocation instance has a $\frac{n}{2n-1}$ -TPS allocation. Moreover, every allocation instance has an allocation that is both $\frac{n}{2n-1}$ -TPS and SEFX.*

For our other main results, we set aside achieving combined share and envy notions guarantees, and focus purely on MMS approximation. As in the case without sellable items, with $n = 3$, MMS cannot be guaranteed.

Proposition 1. *For $n = 3$ agents with additive valuations over $m = 8$ indivisible goods:*

1. *With sellable goods, for every ϵ , there is an allocation instance in which no allocation gives every agent at least $\frac{11}{12} + \epsilon$ of her MMS.*

It is instructive to note that the approximation gap of $\frac{11}{12}$ in Proposition 1 is wider than the currently known MMS approximation gap in the setting without selling, which is $\frac{39}{40}$. Moreover, as the result in [22] shows, for these parameters ($n = 3, m = 8$) it is possible to guarantee the full MMS in the setting without selling, which proves a gap between the two settings.

Given that full MMS cannot always be guaranteed, we consider approximations. The following result is the most technically involved among the results in this paper.

Theorem 3. *For every $n \geq 4$ and every allocation instance, there is a $\frac{2}{3}$ -MMS allocation. For $n = 3$, there is a $\frac{3}{4}$ -MMS allocation.*

The proof of Theorem 3 is based on an approach referred to as “Lone Divider” [36,31,19] that guarantees $(\frac{2}{3} + \Omega(\frac{1}{n}))$ -MMS in the setting without selling [34]. The algorithm works in rounds. In each round it allocates bundles that are acceptable (guarantee at least $\frac{2}{3}$ -MMS) to those agents that receive them, but are unacceptable to each remaining agent i . To prove that eventually agent i gets an acceptable bundle, one uses the fact that bundles allocated in previous rounds have only small value according to v_i , so sufficient value remains in the set of yet unallocated items, enabling the formation of bundles that are acceptable for i . However, with selling, it is not only that bundles were allocated in previous rounds, but also goods were sold. If goods of high value under v_i were sold for a low price, then it may be that goods that remain towards later rounds do not have sufficient value for i to get an acceptable bundle.

Overcoming this difficulty involves multiple new ideas:

1. Handle items of large value in a preliminary phase. This is a standard component in other allocation algorithms, but not needed in the lone divider approach, in the setting without selling.
2. Choose the Lone Divider agent carefully (in our case, as the agent with highest MMS). Without sellable items, any agent can be the divider, but with sellable items, this is not true (as we show even for the $n = 2$ case, see Example 1).

3. Insist on a canonical partition, rather than any partition. In this partition, all bundles have convenient structure, except for at most one (referred to as a “leftovers” bundle).
4. In the matching phase of Lone Divider, we exclude the divider and the leftovers bundle, and add them to the matching when appropriate.

Points 3 and 4 are our major departure from the standard lone divider approach, and may turn out to be useful also in other settings.

With the above ideas and an intricate proof, we recover nearly the same approximation ratio as without sellable items. Specifically, we get $\frac{2}{3}$ -MMS approximation, without the additional $\Omega(\frac{1}{n})$ term. We provide a detailed overview of these ideas in Section 3.4, and the full proof is in Appendix B.

For the important special case of $n = 3$, we are able to fully recover the Lone Divider approximation ratio in the setting without selling, of $\frac{3}{4}$ -MMS. To do so, we combine the principles developed for general n (implementing them becomes somewhat simpler, as n is so small) together with the $n = 2$ full-MMS algorithm, and get an improved $\frac{3}{4}$ -MMS approximation. The details appear in Appendix B.

Beyond our existence results, we are also interested in polytime computation. Without sellable items, it is known that computing the MMS value of a single agent is weakly NP-hard for $n = 2$ and strongly NP-hard for general n . However, a polynomial time approximation scheme (PTAS) for calculating MMS without selling is known, and we show one for our setting as well. We also show that when n is constant, the problem has a pseudo-polynomial algorithm (for both results, see Appendix E). Our PTAS, in turn, means that our existence results for MMS approximation can be appropriately made into a computational result:

Proposition 2. *An allocation as guaranteed by Theorem 1 can be computed in pseudo-polynomial time. Each of our other MMS approximation existence results (Theorem 3) that guarantees ρ -MMS approximation has a PTAS that guarantees $(\rho - \epsilon)$ -MMS.*

For TPS, we show it can be computed in polynomial time, and moreover, the approximation algorithm that incorporates this calculated TPS value runs efficiently. Our proof for combining TPS with SEFX builds on an algorithm that combines TPS approximation with ϵ -SEFX, where the potential argument it provides yields a fully polynomial time approximation scheme (FPTAS).

Proposition 3. *An allocation that guarantees $\frac{n}{2n-1}$ -TPS to each agent can be computed in polynomial time. An FPTAS exists for an allocation that is $\frac{n}{2n-1}$ -TPS and ϵ -SEFX.*

2 Related Work

Table 1 provides an overview of our main results concerning approximate MMS allocations for goods, and compares them with known results without sellable items.

A model with sellable additive goods and $n = 2$ agents is introduced in [30]. In this model, the market price has a specific form that depends on the subjective valuations. Subjective valuations are also normalized so that their total sum is 1. They focus on studying the *price of fairness*, meaning the loss of welfare due to restricting attention to envy-free allocations. Algorithm aspects of the *price of fairness* problem in a model of sellable goods are considered by [12]. The authors show the price of fairness is NP-hard to compute with $n = 2$, give a PTAS for the case of $n = 2$ and identical valuations, and a PTAS and pseudo-polynomial algorithm when agent subjective valuations do not vary too much (i.e., there is a constant bound between the maximal and minimal positive valuations).

	MMS $n = 2$	MMS $n = 3$	MMS $n \geq 4$	TPS ($n \geq 2$)
No Selling	1	$[\frac{11}{12}, \frac{39}{40}]$	$[\frac{7}{9}, 1 - \frac{1}{n^4})$	$\frac{n}{2n-1}$
With Selling	1	$[\frac{3}{4}, \frac{11}{12}]$	$[\frac{2}{3}, 1 - \frac{1}{n^4})$	$\frac{n}{2n-1}$

Table 1. Share approximation results for our new setting of goods with selling, with a comparison to the current state of the art for indivisible goods (without selling). The full MMS guarantee with $n = 2$, and the TPS guarantee, are tight, and we can guarantee that the allocations are SEFX. The No Selling results are due, respectively, to [15], [21], [22], [28], [22], and [7]. The upper bound for $n \geq 4$ with selling, is not a new result, and follows simply because our model generalizes the setting without selling. Other than that, we show the With Selling results, respectively, in Theorem 1), Theorem 3, Proposition 1, Theorem 3, Theorem 2.

A setting in which some goods are divisible and some are indivisible is considered in [9]. In this setting, divisibility is fixed exogenously and does not depend on agents’ preferences. A main difference from our model is that in our model the endogenous selling choice during allocation decides which items will be treated as divisible and which as indivisible. Another is that in our model, once a good is sold, all agents have exactly the same value for the resulting divisible good, which is the sales proceed, and this value is independent (and possibly different) from their subjective values for the unsold good. In contrast, for a divisible good in [9], each agent has a subjective value for it, and moreover, this value is the respective fraction of the subjective value of the agent for the whole good. The work adapts the notions of EF1 and EFX to the mixed setting, naming the new notions EFM and EFXM. For these new notions, if agent j holds part of a divisible good, then agent i must not envy j , and otherwise, the notion behaves like EF1 (or EFX, respectively). Our definitions for SEF1 and SEFX have a similar flavor (though are technically different, as our setting is different). Within the same mixed divisible and indivisible goods model, it can be shown [11] that the MMS approximation of the worst mixed divisible and indivisible instance is not worse than the worst MMS approximation of an indivisible goods instance. A complementary result is an MMS allocation algorithm, which gives an approximation α ranging from the MMS approximation for indivisible goods and up to 1, where α is monotonically increasing in the minimum (among all agents) of the utility of the divisible part relative to the indivisible part.

The notion of *subjective divisibility* is introduced in [10]. In this setting, all items are divisible. However, the valuations of agents are such the each agent views some of the items as indivisible, meaning that she herself gets no value from fractions of the item (though other agents might value fractions of that item), and other items as divisible, meaning that a β -fraction of the item gives a β -fraction of the value of the whole item, for all $0 \leq \beta \leq 1$. In the subjective divisibility setting with $n = 2$ and $n = 3$ agents, a tight $\frac{2}{3}$ -MMS can be guaranteed, and with general n , they show a procedure to guarantee $\frac{1}{2}$ -MMS. Compared to the mixed setting described above, subjective divisibility captures better some difficulties that arise also in our model (some agents want to divide a good, some do not, mirroring a dilemma regarding selling a good that appears in our model), but in some respects differs more from our model, in that sales proceeds can be enjoyed by all agents, whereas fractions of a good can be enjoyed only by agents that view it as divisible.

A model where both subjective valuations and market prices exist for the goods, but where goods are not sold is considered in [8]. Rather, they aim for an allocation that is fair both towards market values, and towards the subjective valuations. I.e., the allocation should be fair both in “objective” terms, and in the agents’ subjective perception. The type of questions and results that arise from this dualistic approach are different than our combined (“monistic”) approach. For example, one

result is that it is impossible to achieve $\frac{1}{n}$ -MMS for the subjective values and EF1 for the objective values, or vice-versa, where the MMS is computed using only the subjective (or only the objective) values. In our approach, the MMS is computed using both subjective and objective values, and we are able to guarantee a good approximation of it.

Another setting that intertwines market values and subjective preferences is considered in [18]. However, the subjective valuation take a specific form of “correction” to the market values, to allow for easy elicitation. The solution concept and discussion are tailored for their setting.

Fair allocation when it is possible to make monetary transfers between the agents is considered in [13]. Introducing this option results in different auction-like mechanisms. Our setting is more limited in that agents do not have “outside money” that can streamline the fair allocation. For example, consider a divorce setting: *All* the partners’ assets are being redistributed, and so, while they can sell existing goods and share the proceeds, they do not have extra-marital money to facilitate compensation payments. Similarly, inheritors / stakeholders in a liquidated company may have outside assets that are negligible compared to the value of assets of the company itself.

In the setting of indivisible goods, the notions of *maximin share* (MMS) and *envy-freeness up to one good* (EF1) are attributed to [15]. A line of works [32,26,25,2,3,27,28] develops allocations with increasingly better MMS approximations, going from $\frac{2}{3}$ to $\frac{7}{9}$. Envy-freeness up to *any* good (EFX) was first introduced in [16]. EFX was shown to exist in several notable settings, such as with $n = 3$ additive agents [17], lexicographic preferences [23], and agents with identical valuations [33].

3 Overview of Proofs

3.1 MMS + SEFX for $n = 2$ Agents

In this section we restate and prove Theorem 1. Our proof is based on a Cut & Give protocol, which is a variation on the well known Cut & Choose protocol. We remark that for the giver in the protocol, we get the stronger EF property, and not just SEFX, and for the cutter, we get EF in interesting special cases.

Theorem 1. *For two agents with additive valuations, there is an allocation that is simultaneously SEFX and MMS.*

Proof. We may assume that in her own MMS partition, each agent sells at most one good (see Lemma 5). We refer to the agent whose sold good has the lower price (or that does not sell a good, and breaking ties arbitrarily) as the *Cutter*, and to the other agent as the *Giver*. We refer to the goods that they intend to sell in their MMS partitions as g_C and g_G , respectively. If one of the agents does not sell a good, or if $g_C = g_G$, then the standard Cut & Choose protocol proves the theorem. So, we assume that $g_C \neq g_G$, with $p(g_C) \leq p(g_G)$.

Let $(A_1, P_1), (A_2, P_2)$ be the MMS partition of Cutter, with $P_1 + P_2 = p(g_C)$, and assume without loss of generality that $g_G \in A_1$. Being her MMS partition, getting any of these two parts also ensures for Cutter the SEFX property. (In fact, if g_C is non-empty, meaning that Cutter really intends to sell a good, she gets EF, due to Lemma 7.)

If at least one of the two parts is acceptable to Giver (gives her at least her MMS value), then Giver takes the part that she prefers, ensuring both MMS and EF for Giver. Cutter gets the other part.

If no part is acceptable for Giver, then she gives (A_2, P_2) to Cutter. However, Giver does not take (A_1, P_1) for herself (and for this reason, we refer to the protocol as Cut & Give, not Cut & Choose). Instead, she sells g_G and cancels the sale of g_C , and keeps for herself (A'_1, P'_1) , with $A'_1 = A_1 \cup \{g_C\} \setminus \{g_G\}$ and $P'_1 = p(g_G) - P_2$. Importantly, $P'_1 \geq 0$, because $p(g_G) \geq p(g_C)$. Giver

gets at least her MMS value, because the total value of the two parts (A'_1, P'_1) and (A_2, P_2) is at least twice her MMS (as they contain exactly the same items and sum of payments as the join of the two parts of her MMS partition), and she did not take the part that has value smaller than her MMS. For the same reason, the EF property is satisfied as well.

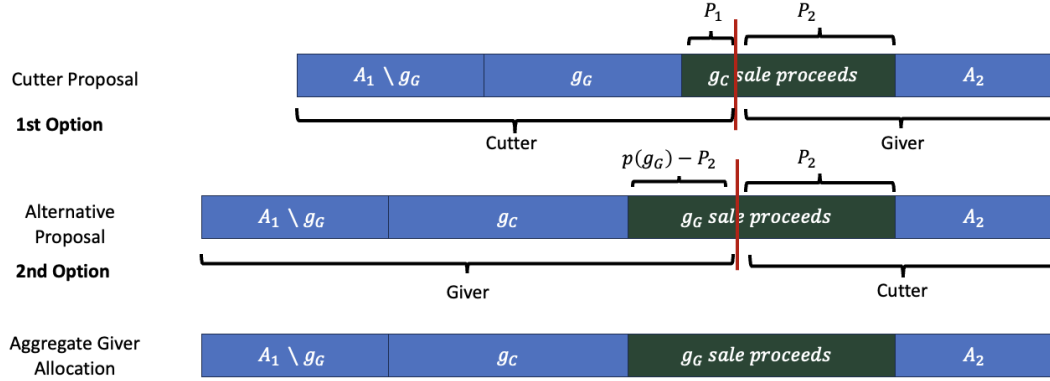


Fig. 1. A visual proof for the $n = 2$ MMS existence result, which follows the outline of the proof of Theorem 1.

3.2 $\frac{n}{2n-1}$ -TPS + SEFX for General n

The TPS (see Definition 3) is efficiently computable, by an algorithm we describe in Appendix D. We know that in the indivisible setting, it is impossible to generally achieve an approximation better than $\frac{n}{2n-1}$ to TPS. This immediately extends to our more general setting as well. We show how a combination of a “moving knife” and bag-filling procedures achieves the optimal $\frac{n}{2n-1}$ -TPS approximation. The algorithm works as follows. First, if there is a good g with a market price higher than $\frac{n}{2n-1} \cdot \text{TPS}_i$ for some agent i , we sell the good and let the agent i with lowest MMS value take $\frac{n}{2n-1} \cdot \text{TPS}_i$ of the received money. We add the remaining sale proceeds from the good back as a virtual “good” (it is already sold, but for the purpose of the moving-knife loop, and later bag-filling, we treat it as one complete good). If we can satisfy another agent using the sale proceeds from at most one single good, we continue to do so (always prioritizing the agent requiring the least amount of sale proceeds), until all agents i and goods g satisfy $p(g) < \frac{n}{2n-1} \text{TPS}_i$. Then, we allocate any item g for which some agent i has $v_i(g) \geq \frac{n}{2n-1} \text{TPS}_i$ to this agent. We then perform a bag-filling procedure into a bag B , where we add an arbitrary good at each step, until some agent has a value of at least $\bar{v}_i(B) \geq \frac{n}{2n-1} \text{TPS}_i$, and then we give the bag to the agent, and repeat. For the full algorithm pseudo-code, see Algorithm 4 in the appendix.

Lemma 2. *In the setting with indivisible sellable goods and additive valuations, the algorithm described above for TPS approximation (with pseudo-code in Algorithm 4) produces a partial allocation that is $\frac{n}{2n-1}$ -TPS.*

In the appendix, we extend the algorithm to produce a (partial) allocation that is both a $\frac{n}{2n-1}$ -TPS approximation and SEFX. We do so using the idea of “stealing” (with relevant precedents in [4,20]), where during the allocation process agents may steal a minimal subset of the bundle that is meant to be allocated to some other agent, if they are envious of it (in the SEFX sense), and release their current bundle. Implementing this process with sellable goods adds complexity to the stealing

process, and to its analysis. One modification that we make is not to implement SEFX directly, but instead implement ϵ -SEFX, and to show that taking the limit as ϵ tends to 0 gives an SEFX allocation. Another principle that we use is that the stealing agent can sell only one item that was not sold in the bundle that she envied. Another aspect that we take care of is money contained in the bundle released by the stealing agent, as this money came from a past selling of some good, and we might need to undo some of the past sales.

Theorem 2. *Every allocation instance has a $\frac{n}{2n-1}$ -TPS allocation. Moreover, every allocation instance has an allocation that is both $\frac{n}{2n-1}$ -TPS and SEFX.*

3.3 $\frac{11}{12}$ -MMS Gap for $n = 3$

Full MMS cannot be guaranteed with $n = 3$ agents. Moreover, our result shows a quantitative gap between the MMS guarantees in the setting without sellable goods, and the one with sellable goods.

Proposition 1. *For $n = 3$ agents with additive valuations over $m = 8$ indivisible goods:*

1. *With sellable goods, for every ϵ , there is an allocation instance in which no allocation gives every agent at least $\frac{11}{12} + \epsilon$ of her MMS.*

Proof. We say that an agent i is *sell-seeking* if $v_i(e) \leq p(e)$ for every good e . For a sell-seeking agent i , her MMS is equal to her proportional share. That is, $MMS_i = \frac{1}{n} \sum_{e \in \mathcal{M}} p(e)$.

For $\epsilon > 0$, we say that an agent i is ϵ -*monetary* if $\sum_{e \in \mathcal{M}} p(e) \leq \epsilon \cdot MMS_i$. If ϵ is sufficiently small, then MMS_i is determined by v_i alone, up to negligible terms.

Suppose that there is one sell-seeking agent i , and all other agents are ϵ -monetary. Then, up to ϵ , a ρ -MMS allocation needs to give each of the ϵ -monetary agents ρ -MMS through undivided goods, while the price of the sold goods needs to be at least $\rho \cdot MMS_i$. This is similar to the standard setting of ρ -MMS allocations for indivisible goods, with the following two changes:

1. For the sell-seeking agent i , the valuation v_i is replaced by the price function p .
2. Agent i needs to get at least ρ times her proportional share, rather than ρ -times MMS.

In [21], it was shown that for three agents, there is an allocation instance with 8 indivisible goods in which in every allocation, either at least one of the first two agents gets at most $\frac{11}{12}$ of her MMS, or the third agent gets at most $\frac{11}{12}$ of her proportional share. Concretely, the example in [21] shows valuations $\tilde{v}_1, \tilde{v}_2, \tilde{v}_3$, so that either $\tilde{v}_1$ or $\tilde{v}_2$ get at most $\frac{11}{12}$ their MMS value, or $\tilde{v}_3$ get at most $\frac{11}{12}$ its proportional value. Moreover, in their instance, the MMS values for the two agents and proportional share for the third agent are all equal. Adjusting to our setting, let $p = \epsilon \cdot \tilde{v}_3, v_1 = \tilde{v}_1, v_2 = \tilde{v}_2, v_3 = 0$. Then, the MMS with selling of agent 3 equals the proportional value of $\tilde{v}_3$, and the MMS with selling of agents 1, 2 equals (up to ϵ) the MMS of $\tilde{v}_1, \tilde{v}_2$. We can assume any good sold goes fully to agent 3, since it is negligible for the other agents. Then, a partition that achieves a better than $\frac{11}{12} + \epsilon$ approximation for our constructed instance, implies a partition that violates the construction in [21].

3.4 A $\frac{2}{3}$ -MMS Algorithm for General n

Theorem 3. *For every $n \geq 4$ and every allocation instance, there is a $\frac{2}{3}$ -MMS allocation. For $n = 3$, there is a $\frac{3}{4}$ -MMS allocation.*

For the general n part of the theorem, we keep ρ as a parameter (rather than fix it throughout to be $\frac{2}{3}$). We assume that $\frac{1}{2} < \rho < 1$.

We recall that the valuation function of agent i is additive and denoted by v_i , the price of good e is denoted by $p(e)$, and we use $\bar{v}_i$ to denote the additive valuation function satisfying $\bar{v}_i(e) = \max[v_i(e), p(e)]$ for every good $e \in \mathcal{M}$.

As we show in Corollary 6, we may assume that each agent i has an MMS partition in which each part has $\bar{v}_i$ value of exactly MMS_i . In this MMS partition, some goods are sold and the money received is split over more than one part. (A good that is sold but its money all belongs to the same part need not be sold at all, as we consider $\bar{v}_i$.) We refer to the set of goods that i desires to sell (and split the money) as $\mathcal{S}_i$.

Our allocation algorithm is inspired by a known algorithm [34,6] that was developed in the case in which goods are not sellable. However, to handle sellable goods, we introduce multiple new components to that algorithm.

Let us first provide a brief overview of the case without sellable goods. Fix a value of ρ that is sufficiently small so that the algorithm does not get stuck, and conditioned on that, as high as possible. A bundle B is said to be *acceptable* to agent i if $v_i(B) \geq \rho \cdot MMS_i$.

An agent is said to be *active* if she did not yet receive any bundle. At a given step of the algorithm, n' denotes the number of active agents.

1. Pick an arbitrary active agent i . Agent i proposes a partition of the remaining goods into n' bundles, each acceptable for i . We refer to such an n' -partition as acceptable.
2. Employ a *matching procedure*. Hall's theorem for bipartite matching can be shown to imply that for some $1 \leq k \leq n'$, we can match k active agents each with a bundle that is acceptable for her, with the additional property that none of the matched bundles is acceptable to any of the remaining agents. Allocate the matched bundles to the respective active agents.

A key aspect in the analysis is to show that for every agent, if $n - n'$ bundles not acceptable to her are removed, the remaining goods still have an n' -partition into acceptable bundles. In the setting without selling, it is known that for this property to hold, the best possible value of ρ is $\frac{2}{3} + \Theta(\frac{1}{n})$.

Adapting the algorithm to the setting with sellable goods raises a major challenge. The fact that a bundle B is not acceptable to agent i no longer implies that by giving B to another agent, the total value of the remaining goods decreases by at most $\rho \cdot MMS_i$. The problem is that B might contain money received from selling a good e . If agent i did not desire to sell e (in particular, $e \notin \mathcal{S}_i$), then in her original MMS partition agent i got value of $v_i(e)$ from good e , but now that e is sold, its value is $p(e)$. If $p(e) < v_i(e)$, the allocation of B caused agent i not only a loss of $\rho \cdot MMS_i$, but also an additional loss of $v_i(e) - p(e)$. Due to this additional loss, the analysis of the above algorithm breaks down.

We now explain some high level ideas that we use in order to cope with this additional loss. To better appreciate our new contributions, it is convenient to note that the $\frac{2}{3}$ bound without sellable goods is a consequence of the following more general statement, that we shall refer to as the *approximate partition guarantee*. For every $\frac{1}{2} \leq \rho \leq \frac{2}{3}$, if $n - n'$ bundles each of value at most $2(1 - \rho)MMS_i$ (or more generally, of total value at most $2(1 - \rho)(n - n')MMS_i$) are removed, then the remaining goods have an n' -partition in which each part has value at least $\rho \cdot MMS_i$.

One principle that we use is to modify the algorithm in a way that allows us to upper bound this additional loss. This principle has three components:

- The first component involves the structure of the n' -partition. Rather than allowing an agent i to pick an arbitrary acceptable n' -partition, we require that for each part by itself, it suffices

to add money from a single good that is sold in order to make the part acceptable (Lemma 5 shows that this property can be enforced). This limits the additional loss from each bundle to at most $v_i(e) - p(e)$ for some element e .

- The second component is to ensure that $v_i(e)$ is not too large. In the case with no sellable goods, we can always assume that $v_i(e) \leq \rho \cdot MMS_i$, because we otherwise give e to agent i , and the MMS of other agents does not decrease. With sellable goods, it is no longer true that by giving e to i the MMS of other agents does not decrease. For example, there may be an agent i' who values all goods other than e at 0 and only derives value from selling e . Consequently, we employ a somewhat more complicated procedure so as to arrive to the situation in which $v_i(e) \leq \rho \cdot MMS_i$ for every i and e . Let i be the agent with smallest MMS_i value for which there is a good e satisfying $p(e) \geq \rho \cdot MMS_i$. If there are such e and i , then sell e , and from the money received give i only $\rho \cdot MMS_i$. (This is analogous to the moving-knife procedure used with divisible goods: giving an agent the smallest part of a good that is still acceptable to her.) The rest of the money that is received can be used by other agents. After all such cases are cleared, if there are such e and i so that $v_i(e) \geq \rho \cdot MMS_i$, let agent i take good e in full.
- The third component is to ensure that $p(e)$ is not too small. For this, we allow agent i to sell a good e only if $p(e) \geq (1 - \rho)MMS_i$. If $p(e) < (1 - \rho)MMS_i$, then good e is required to be contained as a whole in one of the parts of the n' -partition (regardless of whether it was sold in i 's original MMS partition). An aspect that helps absorb the loss incurred by failing to sell e is that it suffices that each part of the n' -partition has value at least $\rho \cdot MMS_i$. To ensure that $p(e) \geq (1 - \rho)MMS_j$ also for other active agents j , at each iteration we choose the partitioning agent i to be the active agent with *highest* MMS value.

Using the above three components, we can upper bound the loss incurred to active agent i due to the forced sale of goods when a bundle B was given to agent j . The first component implies that there was only one sold good e . The second component implies that $v_i(e) \leq \rho \cdot MMS_i$. The third component implies that $p(e) \geq (1 - \rho) \cdot MMS_i$. Hence the loss is upper bounded by $v_i(e) - p(e) \leq \rho \cdot MMS_i - (1 - \rho)MMS_i = (2\rho - 1)MMS_i$.

Another principle that we use in our analysis is the following. We show that we can arrange the partition so that if a removed bundle B causes a loss larger than $\rho \cdot MMS_i$, then B has a particular simple structure. Namely, in this case B is composed of one good allocated in full and part of the money from selling an additional good. To see why this may be helpful, observe that for $\rho = \frac{2}{3}$, the loss of value due to selling a single good is at most $(\rho - (1 - \rho)) \cdot MMS_i = \frac{1}{3} \cdot MMS_i$, and so the combined loss incurred by allocating B to a different agent is at most MMS_i , since in the matching provided by Hall's theorem, the active agents find B unacceptable (i.e., its value is bounded by $\frac{2}{3}MMS_i$). Consider a related (but easier situation) in which B is simply composed of two goods and has total value at most MMS_i . In this case, if both goods belong to the same bundle in the original MMS partition we can simply give up the whole bundle, and if they belong to different bundles, then the union of what remains from these bundles gives a new bundle of value at least MMS_i (because valuations are additive). Hence in either case, the MMS value is not hurt by having some other agent receive B . In our setting, B is composed of one good and some money rather than two goods, and the above local analysis (involving only two bundles) does not work. However, a more global type of analysis can be made to work.

To implement this structural property, we would like the n' -partition proposed by agent i to be composed of only two types of parts. We refer to the first type as *pure* bundles. These bundles do not contain any money from goods that i proposes to sell. For a pure bundle, if it is not acceptable to an agent j , then having it allocated to some other agent causes j a loss of at most $\rho \cdot MMS_i$. We refer to the other type as *singleton* bundles. They contain a single good, and money from selling at

most one other good, as explained in the second principle above. One thing that eases the analysis is allowing one special bundle we call the “leftovers” bundle to have no restriction on its type.

We show that such partitions (that we refer to as *canonical*) do exist. To make use of canonical partitions, we need to ensure that in the matching procedure, we do not end with a partial matching in which some agent receives the leftovers bundle. We ensure this property by modifying the matching procedure. We drop from the bipartite graph G both the leftovers bundle, and the agent i who proposed the n' -partition.

We introduced above two main principles used in our proof. One is to make the additional loss due to sale of goods as small as feasibly possible (unfortunately, we cannot bring it down to 0). The other is that bundles that cause a loss larger than $\rho \cdot MMS_i$ must have a very simple structure. These principles provide a high level understanding of the proof, but in order to obtain a rigorous proof, there are many details to fill in. These are addressed in our full proof. For lack of space, it is presented in the appendix. See Appendix B.

3.5 A $\frac{3}{4}$ -MMS Algorithm for $n = 3$ Agents

We prove two lemmas that allow us to further optimize the $n = 3$ case, and get a $\frac{3}{4}$ approximation.

Lemma 3. *Given an MMS partition for $n = 3$, and where any good g has $\bar{v}_i(g) < \frac{3}{4} \cdot MMS_i$, we can create a canonical form partition of acceptable bundles each of value at least $\frac{3}{4} \cdot MMS_i$.*

Lemma 4. *Suppose that B is either a singleton bundle or a pure bundle, and $\bar{v}_i(B) \leq \frac{3}{4} MMS_i$ holds for agent i . Then if B is allocated to some other agent, agent i can partition the remaining goods into two disjoint bundles B'_1, B'_2 , each of value $\bar{v}_i(B'_j) \geq \frac{3}{4} MMS_i$.*

The two lemmas can be applied through an algorithm that adapts the general n algorithm (for $\frac{2}{3}$ -MMS approximation). The general structure is as follows: (i) If there is a good that an agent values above $\frac{3}{4}$ -MMS, allocate it (or part of its sale proceeds) through a moving-knife procedure. If indeed there is such a good, then we have a single agent that is allocated a bundle they accept, and two remaining agents that do not accept the bundle already allocated. Lemma 4 then ensures that the remaining two agents value the remaining goods as at least $\frac{3}{4}$ -MMS. (ii) If there is no such good, the highest MMS agent creates a canonical partition (made of pure or singleton bundles, and at most one leftovers bundle) with value of at least $\frac{3}{4}$ -MMS for each bundle (as Lemma 3 ensures can be done). (iii) We create a bi-partite graph between the two non-partitioning agents and the two non-leftover bundles in the partition, where an edge between an agent and a bundle exists if the agent finds the bundle acceptable (i.e., values it at least as $\frac{3}{4}$ -MMS). If there is a perfect matching, we assign according to the matching, and assign the leftover bundle to the partitioning agent (denote her agent 1). If there is a bundle in the bi-partite graph that no agent wants, we assign it to the partitioning agent, and apply Lemma 4. The only remaining case is that one of the non-partitioning agents (denote her agent 2) accepts both bundles, and the other (denote her agent 3) finds both unacceptable. If any of the two bundles is pure, then agent 3’s loss from the two bundles being allocated to agent 1 and 2 (which is possible, as they both find both bundles acceptable) is at most $2 \cdot \frac{3}{4}$ -MMS by the bundles themselves (since they are unacceptable to agent 3), and another at most $\frac{3}{4}$ -MMS from $v_3(e) - p(e)$ due to the sale of at most a single good (as there is at most one non-pure bundle among the two). All in all, this implies that the leftover bundle is valued at least as $\frac{3}{4}$ -MMS by agent 3, and can be allocated to her. Then, the only remaining subcase is where both non-leftover bundles are singleton bundles. In this case, it is w.l.o.g. to assume that agent 1 values them exactly at $\frac{3}{4}$ -MMS (as any superfluous sale proceeds can be moved to the leftover bundles). Then, we can allocate one of the singleton bundles to agent 2, and apply Lemma 4 to agent 1 and 3.

In all events that we invoke Lemma 4, we thereafter use our $n = 2$ algorithm (Theorem 1) to output an allocation that gives each of these agents at least $\frac{3}{4}$ -MMS.

4 Discussion

In the appendices, we consider two natural extensions of our model.

4.1 Equal Proceeds

For our discussion so far, we have considered *unrestricted* proceeds distribution, where the sale proceeds may be arbitrarily distributed among the agents. An interesting case is *equal* proceeds distribution, where each agent must receive an equal monetary compensation:

$$\forall i, P_i = \frac{1}{n} \sum_{g_j \in \text{sellable}} p(g_j).$$

Clearly, with this requirement, the MMS value of agents is weakly smaller than without this requirement. We show an example where only a $\frac{1}{n}$ approximation to this weaker MMS notion can be guaranteed. We also show an example where this weaker MMS notion guarantees only $\frac{1}{n}$ of the MMS notion with unrestricted proceeds. We complement the above negative results with an algorithm that guarantees $\frac{1}{n}$ -TPS (as defined for the unrestricted proceeds) to each agent.

4.2 Chores with Outsourcing

We consider allocation of chores (items of negative value that must be allocated). With chores, instead of the option of selling, agents have the option of outsourcing, where the agents collectively need to pay the market-price of outsourced chores (and then, split the payment in some way among the agents). Our results for MMS approximation for chores mirror in spirit our results for goods. Namely, we find that outsourcing adds difficulties to the design of allocation algorithms, and show approximation gaps that are wider than those known without outsourcing. We design allocation algorithms with constant approximation of the MMS, but not as good a constant as is known without outsourcing.

Theorem 4. *Every allocation instance with chores (and optional outsourcing) has a 2 -MMS allocation. For every $\epsilon > 0$, there are allocation instances that do not have $(\frac{19}{18} - \epsilon)$ -MMS allocations.*

LLM Usage Disclosure

When finalizing the manuscript, we were assisted by the AI feedback tool Refine.ink. We have also used the auto-formalizing system Aristotle [1] to proofread the paper and produce a Lean4 formalization of the manuscript, available on .

Acknowledgments

This research was supported in part by the Israel Science Foundation (grant No. 1122/22).

References

1. Achim, T., Best, A., Bietti, A., Der, K., Fédérico, M., Gukov, S., Halpern-Leistner, D., Henningsgard, K., Kudryashov, Y., Meiburg, A., Michelsen, M., Patterson, R., Rodriguez, E., Scharff, L., Shanker, V., Sicca, V., Sowrirajan, H., Swope, A., Tamas, M., Tenev, V., Thomm, J., Williams, H., Wu, L.: Aristotle: Imo-level automated theorem proving (2025), <https://arxiv.org/abs/2510.01346>
2. Akrami, H., Garg, J., Sharma, E., Taki, S.: Simplification and improvement of mms approximation. In: Elkind, E. (ed.) Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23. pp. 2485–2493. IJCAI ’23, International Joint Conferences on Artificial Intelligence Organization, Macao, P.R.China (8 2023). <https://doi.org/10.24963/ijcai.2023/276>, <https://doi.org/10.24963/ijcai.2023/276>, main Track
3. Akrami, H., Garg, J., Sharma, E., Taki, S.: Improving approximation guarantees for maximin share. In: Proceedings of the 25th ACM Conference on Economics and Computation. p. 198. EC ’24, Association for Computing Machinery, New York, NY, USA (2024). <https://doi.org/10.1145/3670865.3673544>, <https://doi.org/10.1145/3670865.3673544>
4. Akrami, H., Rathi, N.: Achieving maximin share and efx/efl guarantees simultaneously. In: Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence. AAAI’25/IAAI’25/EAAI’25, AAAI Press (2025). <https://doi.org/10.1609/aaai.v39i13.33477>, <https://doi.org/10.1609/aaai.v39i13.33477>
5. Alon, N., Spencer, J.H.: The probabilistic method. John Wiley & Sons, Hoboken, NJ (2016)
6. Amanatidis, G., Markakis, E., Nikzad, A., Saberi, A.: Approximation algorithms for computing maximin share allocations. ACM Trans. Algorithms **13**(4) (Dec 2017). <https://doi.org/10.1145/3147173>, <https://doi.org/10.1145/3147173>
7. Babaioff, M., Ezra, T., Feige, U.: On best-of-both-worlds fair-share allocations. In: Web and Internet Economics: 18th International Conference, WINE 2022, Troy, NY, USA, December 12–15, 2022, Proceedings. p. 237–255. Springer-Verlag, Berlin, Heidelberg (2022). https://doi.org/10.1007/978-3-031-22832-2_14, https://doi.org/10.1007/978-3-031-22832-2_14
8. Barman, S., Ebadian, S., Latifian, M., Shah, N.: Fair division with market values. In: Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence. AAAI’25/IAAI’25/EAAI’25, AAAI Press (2025). <https://doi.org/10.1609/aaai.v39i13.33484>, <https://doi.org/10.1609/aaai.v39i13.33484>
9. Bei, X., Li, Z., Liu, J., Liu, S., Lu, X.: Fair division of mixed divisible and indivisible goods. Artificial Intelligence **293**, 103436 (2021)
10. Bei, X., Liu, S., Lu, X.: Fair division with subjective divisibility. Games and Economic Behavior **151**, 127–147 (2025). <https://doi.org/https://doi.org/10.1016/j.geb.2025.03.004>, <https://www.sciencedirect.com/science/article/pii/S0899825625000417>
11. Bei, X., Liu, S., Lu, X., Wang, H.: Maximin fairness with mixed divisible and indivisible goods. Autonomous Agents and Multi-Agent Systems **35**(2), 34 (2021). <https://doi.org/10.1007/s10458-021-09517-7>
12. Bilò, V., Markakis, E., Vinci, C.: Achieving Envy-Freeness Through Items Sale. In: Chan, T., Fischer, J., Iacono, J., Herman, G. (eds.) 32nd Annual European Symposium on Algorithms (ESA 2024). Leibniz International Proceedings in Informatics (LIPIcs), vol. 308, pp. 26:1–26:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2024). <https://doi.org/10.4230/LIPIcs.ESA.2024.26>, <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2024.26>
13. Bogomolnaia, A., Moulin, H.: Fair division with money and prices: bid & sell versus divide & choose. Theory and Decision **99**(1), 407–443 (2025)
14. Bouveret, S., Lemaître, M.: Characterizing conflicts in fair division of indivisible goods using a scale of criteria. In: Proceedings of the 2014 International Conference on Autonomous Agents and Multi-Agent Systems. p. 1321–1328. AAMAS ’14, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC (2014)
15. Budish, E.: The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes. Journal of Political Economy **119**(6), 1061–1103 (2011)
16. Caragiannis, I., Kurokawa, D., Moulin, H., Procaccia, A.D., Shah, N., Wang, J.: The unreasonable fairness of maximum nash welfare. ACM Transactions on Economics and Computation (TEAC) **7**(3) (sep 2019). <https://doi.org/10.1145/3355902>, <https://doi.org/10.1145/3355902>
17. Chaudhury, B.R., Garg, J., Mehlhorn, K.: Efx exists for three agents. J. ACM **71**(1) (Feb 2024). <https://doi.org/10.1145/3616009>, <https://doi.org/10.1145/3616009>
18. Dall’Aglio, M.: Fair division of goods in the shadow of market values. European Journal of Operational Research **307**(2), 785–801 (2023). <https://doi.org/https://doi.org/10.1016/j.ejor.2022.10.025>, <https://www.sciencedirect.com/science/article/pii/S037722172200813X>

19. Dawson, C.B.: An algorithmic version of kuhn’s lone-divider method of fair division. *Missouri Journal of Mathematical Sciences* **13**(1), 37–44 (2001)
20. Feige, U.: The residual maximin share (2025), <https://arxiv.org/abs/2505.19961>
21. Feige, U., Norkin, A.: Improved maximin fair allocation of indivisible items to three agents (2022), <https://arxiv.org/abs/2205.05363>
22. Feige, U., Sapir, A., Tauber, L.: A tight negative example for mms fair allocations. In: *Web and Internet Economics: 17th International Conference, WINE 2021, Potsdam, Germany, December 14–17, 2021, Proceedings*. p. 355–372. Springer-Verlag, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-030-94676-0_20, https://doi.org/10.1007/978-3-030-94676-0_20
23. Gafni, Y., Huang, X., Lavi, R., Talgam-Cohen, I.: Unified fair allocation of goods and chores via copies. *ACM Trans. Econ. Comput.* **11**(3–4) (Dec 2023). <https://doi.org/10.1145/3618116>, <https://doi.org/10.1145/3618116>
24. Gal, Y., Mash, M., Procaccia, A.D., Zick, Y.: Which is the fairest (rent division) of them all? *Journal of the ACM (JACM)* **64**(6), 1–22 (2017)
25. Garg, J., Taki, S.: An improved approximation algorithm for maximin shares. In: *Proceedings of the 21st ACM Conference on Economics and Computation*. p. 379–380. EC ’20, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3391403.3399526>, <https://doi.org/10.1145/3391403.3399526>
26. Ghodsi, M., Hajiaghayi, M., Seddighin, M., Seddighin, S., Yami, H.: Fair allocation of indivisible goods: Improvements and generalizations. In: *Proceedings of the 2018 ACM Conference on Economics and Computation*. p. 539–556. EC ’18, Association for Computing Machinery, New York, NY, USA (2018). <https://doi.org/10.1145/3219166.3219238>, <https://doi.org/10.1145/3219166.3219238>
27. Heidari, E., Kaviani, A., Seddighin, M., Shahrezaei, A.: Improved maximin share guarantee for additive valuations. In: *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. pp. 2239–2290. Society for Industrial and Applied Mathematics, Philadelphia, PA (2026). <https://doi.org/10.1137/1.9781611978971.81>, <https://epubs.siam.org/doi/abs/10.1137/1.9781611978971.81>
28. Huang, X., Zhou, S.: An fptas for 7/9-approximation to maximin share allocations (2025), <https://arxiv.org/abs/2511.13056>
29. Igarashi, A., Yokoyama, T.: Kajibuntan: A house chore division app. *Proceedings of the AAAI Conference on Artificial Intelligence* **37**(13), 16449–16451 (Jul 2024). <https://doi.org/10.1609/aaai.v37i13.27075>, <https://ojs.aaai.org/index.php/AAAI/article/view/27075>
30. Karp, J., Kazachkov, A., Procaccia, A.: Envy-free division of sellable goods. *Proceedings of the AAAI Conference on Artificial Intelligence* **28**(1) (Jun 2014). <https://doi.org/10.1609/aaai.v28i1.8815>, <https://ojs.aaai.org/index.php/AAAI/article/view/8815>
31. Kuhn, H.W.: On games of fair division. In: *Proceedings of Symposia in Applied Mathematics*. vol. 14. American Mathematical Society (1967)
32. Kurokawa, D., Procaccia, A.D., Wang, J.: Fair enough: Guaranteeing approximate maximin shares. *J. ACM* **65**(2) (Feb 2018). <https://doi.org/10.1145/3140756>, <https://doi.org/10.1145/3140756>
33. Plaut, B., Roughgarden, T.: Almost envy-freeness with general valuations. *SIAM Journal on Discrete Mathematics* **34**(2), 1039–1068 (2020). <https://doi.org/10.1137/19M124397X>, <https://doi.org/10.1137/19M124397X>
34. Procaccia, A.D., Wang, J.: Fair enough: Guaranteeing approximate maximin shares. In: *Proceedings of the fifteenth ACM conference on Economics and computation*. pp. 675–692 (2014)
35. Segal-Halevi, E.: Fair division of land. Ph.D. thesis, Ph. D. Dissertation. Bar Ilan University, Computer Science Department (2017)
36. Steinhaus, H.: The problem of fair division. *Econometrica* **16**, 101–104 (1948)
37. Woeginger, G.J.: A polynomial-time approximation scheme for maximizing the minimum machine completion time. *Oper. Res. Lett.* **20**(4), 149–154 (May 1997). [https://doi.org/10.1016/S0167-6377\(96\)00055-7](https://doi.org/10.1016/S0167-6377(96)00055-7), [https://doi.org/10.1016/S0167-6377\(96\)00055-7](https://doi.org/10.1016/S0167-6377(96)00055-7)

A Missing Proofs and Additional Results for Section 3.1

We start by making the following useful distinction. Instead of talking about an allocation A_i of kept goods and sale proceeds P_i , we convert them to: (i) An allocation B_i of goods that belong in full to the agent, where the agent may decide whether to keep or sell them. For any such good g , the agent receives value $\bar{v}_i(g)$. (ii) Sale proceeds P'_i out of goods that are *sold* (a forced sale, without leaving a choice to the agents about it). For simplicity, we sometimes refer to the bundle of full items, together with sale proceeds from forced-sale goods, as a unified bundle.

The following lemma shows that we can achieve the MMS value with an MMS output with at most $n - 1$ *sold* goods.

Lemma 5. *There always exists an MMS output with n parts has at most $n - 1$ forced-sale goods.*

Proof. Let ℓ be the minimal amount of split goods $g_{i_1}, \dots, g_{i_\ell}$ in an MMS output for the agent. Assume $\ell \geq n$. It must be that

$$\max\{P^1, \dots, P^n\} < \min_{1 \leq j \leq \ell} \{p(g_{i_j})\}, \quad (6)$$

or we could include the good that violates this condition in full in the respective allocation. I.e., if w.l.o.g. we assume that good i_1 violates the condition w.r.t. P^1 , we have $P_1 \geq p(g_{i_1}) = \sum_{i=1}^n P_i^1$, and so we can consider an MMS output where the same goods are sold, and has $\tilde{P}_1 = P_1, \dots, \tilde{P}_n = P_n$, but has:

$$\tilde{P}_1^1 = \sum_{i=1}^n P_i^1.$$

This implies that each other sold good sale proceeds $i_2, \dots, i_\ell$ has $\Delta_{i_j} = P_1^j - \tilde{P}_1^j \geq 0$ in sale proceeds to distribute among $P_2, \dots, P_n$, but by sum conservation this can be done, resulting in an MMS output with the same value but with only $\ell - 1$ fractional good sale proceeds, contradicting ℓ minimality.

It is sometimes convenient to consider that the MMS output has the same value for the agent across all bundles. We show this can be assumed.

Definition 5. *We say an MMS output $B_1, \dots, B_n$ is equi-valued if $\bar{v}_i(B_j) = \bar{v}_i(B_{j'})$ for any j, j' .*

Lemma 6 (Value Readjustment). *For the purpose of guaranteeing MMS approximation, it is w.l.o.g. to assume every agent has an equi-valued MMS partition.*

Proof. Consider agent i and some MMS partition $B_1, \dots, B_n$. Assume some bundle has $\bar{v}_i(B_j) > MMS_i$. This bundle cannot contain sale proceeds. If it does, we could redistribute them equally between B_j and any bundle that has value of exactly MMS_i , and so increase the MMS value. We conclude that we can readjust v_i (without changing p) so that all bundles have the same value in the MMS partition. Let the new subjective valuations be $\hat{v}_1, \dots, \hat{v}_n$. It holds that $\forall 1 \leq i \leq n, g_j \in \mathcal{M}, \hat{v}_i(g_j) \leq v_i(g_j)$, and $MMS(\mathcal{M}, v_i, p, n) = MMS(\mathcal{M}, \hat{v}_i, p, n)$. If we can guarantee $\rho MMS(\mathcal{M}, \hat{v}_i, p, n)$ to every agent i , then we can also guarantee $\rho MMS(\mathcal{M}, v_i, p, n)$ to every agent i : Take the allocation that guarantees $\rho MMS(\mathcal{M}, \hat{v}_i, p, n)$. Then, it also guarantees $\rho MMS(\mathcal{M}, v_i, p, n)$ (as they are equal for each agent). Moreover, the value of the allocation is at least as high under v_i than under $\hat{v}_i$.

It is technically convenient to attribute sale proceeds to goods. I.e., while Eq. 2 maintains that the sum of distributed sale proceeds does not exceed the sum of sold goods, we could introduce a more granular notation P_i^j , where $1 \leq i \leq n$ denotes the agent, and g_j with $1 \leq j \leq m$ denotes the sold good. We require

$$\begin{aligned} \forall j \in \mathcal{S}, \quad \sum_{i=1}^n P_i^j &\leq \xi(g_j) \cdot p(g_j), \\ \forall i \in \mathcal{N}, j \in \mathcal{S}, \quad P_i^j &\geq 0. \end{aligned} \quad (7)$$

Clearly, Eq. 7 implies Eq. 2 by summing over all goods j , but we can also always find a granular representation, e.g., by greedily attributing sale proceeds to the goods sold in ascending order of the agents.

Lemma 7. *With $n = 2$ agents, if agent i has a forced-sale good split between their two MMS output bundles, then both bundles have the same utility for the agent.*

Proof. Let the MMS partition be the unified bundles C^1, C^2 , and assume towards contradiction $u_i(C^2) < u_i(C^1)$. Let $\delta = u_i(C^1) - u_i(C^2)$. There is a split good g between the bundles, where $P^1 + P^2 = p(g)$, and $\min\{P^1, P^2\} > 0$, so that P^1 belongs to the unified bundle C^1 , and P^2 to C^2 . If instead we set the sale proceeds to be $\tilde{P}^1 = \max\{\frac{P^1}{2}, P^1 - \frac{\delta}{2}\}$ in C^1 , and $p(g) - \tilde{P}^1$ in C^2 , the MMS value would increase, in contradiction.

To highlight the contribution of our $n = 2$ MMS existence result, we define a natural extension of the Cut & Choose algorithm to our setting. We show an example where this extension of Cut & Choose fails to produce an MMS allocation, regardless of the agent that cuts.

Definition 6. *Cut & Choose for $n = 2$ agents. Consider two agents i and j . Let C be an MMS partition of agent i . Allocate agent j its preferred (unified) bundle among C^1, C^2 , and allocate the other bundle to agent i . Then, we call this the Cut & Choose allocation where agent i is the “cutting” agent and agent j is the “choosing” agent.*

Example 1. Consider $n = 2$ agents and $m = 5$ goods, with valuations and market price as given in Table 2. There is a left-right mirror symmetry between the agents, so we make the argument for the case Agent 1 cuts and Agent 2 chooses, but the other case is similar. The following is an MMS partition for agent 1: The agent sells g_5 , and allocates one bundle with g_1, g_2 and $\frac{3}{4}$ in sale proceeds, and another bundle with g_3, g_4 and $\frac{1}{4}$ in sale proceeds. When $x > \frac{3}{2}$, this MMS partition is unique. In the perspective of Agent 2, both bundles have a value of $\frac{x}{2} + \frac{5}{4}$. When $x \rightarrow \infty$, this yields a $\frac{1}{2} + \epsilon$ approximation to the MMS with arbitrarily small ϵ .

	g_1	g_2	g_3	g_4	g_5	MMS
Agent 1	x	$\frac{1}{2}$	$\frac{x+1}{2}$	$\frac{x+1}{2}$	0	$x + \frac{5}{4}$
Agent 2	0	$\frac{x+1}{2}$	$\frac{x+1}{2}$	$\frac{1}{2}$	x	$x + \frac{5}{4}$
Market price p	1	0	0	0	1	—

Table 2. Subjective valuations and market prices for Example 1

In fact, the $\frac{1}{2}$ approximation of our example is tight. The example shows a case where no matter which agent cuts, Cut & Choose does not guarantee more than $\frac{1}{2}$ the MMS, whereas the following lemma holds in any case and regardless of the choice of the cutting agent.

Lemma 8. *With $n = 2$, Cut & Choose gives a $\frac{1}{2}$ -MMS approximation.*

Proof. W.l.o.g. assume agent 1 is the cutter. If their MMS partition does not force the sale of any good, then the chooser gets at least their proportional value (and so, also their MMS) by choosing their preferred bundle, and agent 1 gets their MMS value from the remaining bundle.

If agent 1’s MMS partition does force the sale of a good, by Lemma 5 we can assume it is a single good g . If agent 2 also sells g in their MMS partition, then the same argument as before holds. If they do not, then g is contained in full in one of the MMS bundles of agent 2, say C^1 . We have

$$\bar{v}_2(\mathcal{M} \setminus \{g\}) \geq \bar{v}_2(C^2) \geq MMS_2.$$

By choosing their preferred bundle among the partition by agent 1, agent 2 gets at least $\frac{1}{2}\bar{v}_2(\mathcal{M} \setminus \{g\})$, since all goods but g are allocated in full and agent 2 has the discretion of whether to sell them or keep them. Agent 1 gets the remaining bundle and their MMS value.

B Full Proof of $\frac{2}{3}$ -MMS Approximation

We now present the full proof of the general n part of Theorem 3. It uses notation (such as S_i and $\bar{v}_i$) and terminology (such as acceptable bundles) that is introduced in the main text overview.

Proof. In the course of the allocation algorithm, we shall allocate bundles to some agents. We trace the total value of goods (and money received from sold goods) that remains for those agents that are still active (did not yet receive a bundle). For each allocated bundle B , we shall make sure that the loss for any active agent i is of one of four types.

Definition 7 (Loss Types).

- Loss of type ℓ_1 . *The bundle B contains only one good e and no money, and $e \notin S_i$.*
- Loss of type ℓ_2 . *$\bar{v}_i(B) \leq \rho \cdot MMS_i$, and B does not contain money received from selling any good e with $v_i(e) > p(e)$, unless $e \in S_i$ (in which case i also intended to sell e)*
- Loss of type ℓ_3 . *The value of B is derived from two goods, e and f , each with $\bar{v}_i$ value at most $\rho \cdot MMS_i$. Both e and f are not goods in S_i (i.e., agent i would not have chosen to sell them), but the allocation algorithm did sell f , and so B contains e and part of the money received from selling f . The combined loss (B being allocated to another agent, together with forcing the selling of f) is at most $\bar{v}_i(B) + v_i(f) - p(f) \leq \rho \cdot MMS_i + v_i(f) - (1 - \rho)MMS_i = v_i(f) + (2\rho - 1)MMS_i$. (For $\rho = \frac{2}{3}$, this implies that the amortized loss is at most $v_i(f) + (2\rho - 1)MMS_i \leq (\frac{2}{3} + \frac{1}{3})MMS_i = MMS_i$.)*
- Loss of type ℓ_4 . *The value of B is derived from two goods, e and f , each with $\bar{v}_i$ value at most $\rho \cdot MMS_i$. We have that $e \in S_i$, $f \notin S_i$, but f was sold and e was not. B contains e and part of the money received from selling f . The combined loss (losing B in combination of selling f at a loss) is at most $v_i(f) + (2\rho - 1)MMS_i$.*

The allocation algorithm starts with a preliminary phase whose goal is to make sure that $\bar{v}_i(e) \leq \rho \cdot MMS_i$ for every good e and agent i .

The preliminary phase has two stages.

Stage 1.

As long as there is an active agent i and good e with $p(e) \geq \rho \cdot MMS_i$, do the following. Let j denote the active agent with lowest MMS value, and observe that necessarily, there is such a good e with $p(e) \geq \rho \cdot MMS_j$. Sell e , pay $\rho \cdot MMS_j$ to j , and remove j .

We track the loss for the remaining active agents. For any remaining active agent i for which $e \notin S_i$ the loss is of type ℓ_1 , whereas for any remaining active agent i for which $e \in S_i$ the loss is of type ℓ_2 .

When Stage 1 is done, we move to Stage 2.

Stage 2.

If there is any agent i and good e with $v_i(e) \geq \rho \cdot MMS_i$, give e to i , and remove i .

Again, for any remaining active agent j for which $e \notin S_j$ the loss is of type ℓ_1 , whereas for any remaining active agent j for which $e \in S_j$ the loss is of type ℓ_2 (because we have already exhausted Stage 1, and so $p(e) \leq \rho \cdot MMS_j$).

The outcome of the preliminary phase is summarized in the following proposition.

Proposition 4. *After the preliminary phase, $\bar{v}_i(e) < \rho \cdot MMS_i$ for every active agent i and remaining good e .*

We now reach the main part of our algorithm. At this point, there are n' active agents, some goods may still remain, and some money P' may remain (from goods sold in previous steps).

Consider the active agent i of highest MMS value. Find a partition (into n' parts) with the following properties. We use the notation $\mathcal{S}_i$ for the set of goods that i intends to sell, so that the money received from selling any single good is split over more than a single part. Let $P = \sum_{e \in \mathcal{S}_i} p(e)$. Note that P' is not considered part of P .

1. $p(e) \geq (1 - \rho)MMS_i$ for every $e \in \mathcal{S}_i$.
2. Each part is of one of three types:
 - (a) Pure parts: the $\bar{v}_i$ value of a pure part is at least $\rho \cdot MMS_i$, and it contains no money from P . (It may contain money from P' .)
 - (b) Singleton parts: a singleton part contains a single good and some money from P , and its $\bar{v}_i$ value is at least $\rho \cdot MMS_i$. Importantly, for each singleton part, there is an associated good in $\mathcal{S}_i$ such that all the money in the part comes from the sale of the associated good. The same good in $\mathcal{S}_i$ can be associated with several singleton parts.
 - (c) Special part: a part that is neither pure nor singleton. Importantly, there is at most one part that is special in the partition.
3. For every good e_1 in a singleton part and for every good $e_2 \in \mathcal{S}_i$, we have that $\bar{v}_i(e_1) + p(e_2) > \rho \cdot MMS_i$.

We refer to partitions as above as *canonical*. We remark that we do not require canonical partitions to make use of all goods (it is fine if some goods are neither in any of the parts nor in $\mathcal{S}_i$).

Suppose that a canonical partition is found. Consider the following $n' - 1$ by $n' - 1$ bipartite graph G . On one side, we have all active agents except for i . On the other side, we have all parts of the canonical partition (where each part includes the goods of that part and the money in that part, both that received from P' and that received from P), except for the special part. (For uniformity of the presentation, if there is no special part, then remove some other arbitrary part.) We place an edge between an agent j and a part k if part k is acceptable under $\bar{v}_j$.

Given G , we proceed with a *matching procedure*.

1. If G has a perfect matching, we allocate parts according to the matching, and give the special part to agent i . All agents receive acceptable bundles and we are done with the allocation.
2. If G has no edge, we allocate one of the non-special parts to agent i and remove agent i . If this non-special part was a singleton, then we sell a single good from $\mathcal{S}_i$ in order to produce the payment associated with the part, but do not sell the remaining goods from $\mathcal{S}_i$.
3. If none of the above two cases hold, then by Hall's theorem, there is a subset of $t < n' - 1$ parts that can be matched to t active agents, and except for agent i , no unmatched active agent finds any of the matched parts acceptable. Allocate the matched parts to the respective agents and remove these agents.
 - If the subgraph of G that remains still contain an edge, then repeat step 3. After finitely-many repetitions the subgraph necessarily becomes empty. At that point we are at step 2 above, and then agent i gets matched to one of the non-special parts.

When step 3 ends, the payments associated with the matched singleton parts are generated by selling the fewest possible goods from $\mathcal{S}_i$. As the partition is canonical, the number of goods sold does not exceed the number of singleton parts that are allocated.

At the end of the above matching procedure, at least agent i (and perhaps also other agents) receives an acceptable bundle. Hence the number of active agents strictly decreases, meaning that the allocation algorithm eventually ends (of course, we still need to show that we can find canonical partitions at each iteration).

ALGORITHM 1: APX-MMS-2/3

Input: Valuations $v_1, \dots, v_n$ over goods $\mathcal{M}$, market-price p , number of agents n

Output: An allocation B with at least $\frac{2}{3} \cdot MMS$ value for each agent

```

1  $ACTIVE = [n], AVAIL = \mathcal{M}, P' = 0$ 
2 while  $\exists i \in ACTIVE, g \in AVAIL, p(g) \geq \rho MMS_i$  do
3   | Let  $i$  be the lowest MMS agent in ACTIVE.
4   | Sell  $g$ , allocate  $\rho MMS_i$  in sale proceeds to agent  $i$ , add any remaining sale proceeds to  $P'$ . Remove  $g$  from
   | AVAIL, and  $i$  from ACTIVE.
5 end
6 while  $\exists i \in ACTIVE, g \in AVAIL, v_i(g) \geq \rho MMS_i$  do
7   | Give  $g$  to agent  $i$ . Remove  $g$  from AVAIL, and  $i$  from ACTIVE.
8 end
9 while  $|ACTIVE| > 0$  do
10  | Let  $i$  be the highest MMS agent in ACTIVE.
11  | Let  $B_1, \dots, B_{|ACTIVE|}$  be a canonical partition of AVAIL,  $P'$ , where  $B_1$  is the leftovers bundle.
12  | Let  $G$  be the bipartite graph between  $B_2, \dots, B_{|ACTIVE|}$  and the agents in  $ACTIVE \setminus \{i\}$ , with an edge
   | whenever an agent finds a bundle acceptable.
13  | Find a perfect matching, and if not, a maximal matching from a violating set due to Hall's theorem. Denote
   | the matching  $\mu$ . Allocate according to the matching. Remove matched bundles from AVAIL and matched
   | agents from ACTIVE, sell goods not contained in full in a bundle, and adjust  $P'$  accordingly.
14  | if a perfect matching was found then
15  |   | Allocate  $B_1$  to agent  $i$ , remove agent  $i$  from ACTIVE.
16  | end
17  | else
18  |   | Allocate a bundle  $B_j$  from the violating set that was not matched to an agent, to agent  $i$ . Remove  $B_j$ 
   |   | from AVAIL and agent  $i$  from ACTIVE.
19  | end
20 end
```

We now analyze the loss of an agent k that is still active (was not matched). For every pure part that was allocated, the loss is of type ℓ_2 (as the part was not acceptable under $\bar{v}_k$). For the singleton parts, there are several cases to consider. Let t denote the number of singleton parts (numbered from 1 to t) that were matched, and let $e_1, \dots, e_t$ denote the t goods contained in the t singleton parts (the goods that are not sold). Let t' denote the number of goods from $\mathcal{S}_k$ that were sold in order to generate the payments in the singleton parts, and denote these sold goods by $f_1, \dots, f_{t'}$. Recall that necessarily, $t' \leq t$.

We iterate over the singleton bundles to map them into loss events. Each singleton bundle is comprised of a pair e_j, f_j , where e_j is kept, and some sale proceeds from f_j are used. If $f_j \in \mathcal{S}_k$, we consider it a loss of type ℓ_2 . This is because the payment taken from $p(f_j)$ so as to make the singleton part acceptable is such that $\bar{v}_k(e_j) + p_j \leq \rho \cdot MMS_k$.

If $f_j \notin \mathcal{S}_k$, we consider whether this is the first time that f_j has appeared during our iteration over the singleton bundles. If it is, then note that $p(f_j) \geq (1 - \rho)MMS_i \geq (1 - \rho)MMS_k$ (here we use the fact that i , the agent who gets to propose the partition, has the highest MMS value). Hence the fact that f_j was sold caused agent k an amortized loss of at most $v_k(f_j) - (1 - \rho)MMS_k$. The total loss includes also the value of the singleton part B_j containing e_j that was allocated, which is at most ρMMS_k (because B_j was not acceptable for agent k). Hence the total loss is at most $v_k(f_j) + (2\rho - 1)MMS_k$. If $e_j \notin \mathcal{S}_k$, then we have a loss of type ℓ_3 . If $e_j \in \mathcal{S}_k$, then we have a loss of type ℓ_4 .

If it is not the first time that f_j appears in a singleton bundle during our iteration, we consider it a type ℓ_2 loss. This is since agent k does not find the bundle acceptable, e_j is not sold, and the selling loss from f_j was already accounted for in the first singleton bundle that included sale proceeds from f_j .

So far, we established the following proposition.

Proposition 5. *For every active agent i that remains, each agent that was removed received a bundle that caused agent i to suffer a loss of one of the four types ℓ_1 , ℓ_2 , ℓ_3 or ℓ_4 .*

To show that the allocation algorithm terminates with all agents receiving acceptable bundles, we need to set the value of ρ to be such that a canonical partition exists in every step of the main part of the algorithm.

Lemma 9. *For $\rho = \frac{2}{3}$, a canonical partition exists in every step of the main part of the algorithm.*

Proof. Consider active agent i when n' agents remain.

Let $B_1, \dots, B_n$ denote the original MMS_i partition of $\mathcal{M}$, and recall that $\mathcal{S}_i$ is the set of goods that i intends to sell. With the goods and money that remains, we need to produce a ρ - MMS_i partition $B'_1, \dots, B'_{n'}$ that is canonical.

Let n_1, n_2, n_3, n_4 (with $n_1 + n_2 + n_3 + n_4 = n - n'$) be the number of events of losses of types $\ell_1, \ell_2, \ell_3, \ell_4$, respectively.

In losses of type ℓ_1 , a single good $e \notin \mathcal{S}_i$ is sold. Remove the (at most) n_1 bundles that contain these goods. We remain with an MMS_i partition into at least $n - n_1$ parts, and there are no events of ℓ_1 losses.

In a loss event j of type ℓ_3 , two goods e_j and f_j are lost, with $e_j, f_j \notin \mathcal{S}_i$. In addition some money r_j is received, so that the total net loss is at most $(3\rho - 1)MMS_i$. For our choice of $\rho = \frac{2}{3}$, this loss is at most MMS_i . If e_j and f_j belong to the same bundle in $B_1, \dots, B_n$, simply remove that bundle. If they belong to different bundles, then after removing e_j and f_j , merge what remains of these two bundles, and add to the newly created bundle the payment of r_j . Handling all loss events of type ℓ_3 , we remain with an MMS_i partition into at least $n - n_1 - n_3$ parts, and there are no events of ℓ_1 or ℓ_3 losses.

As we may discard parts, we assume that exactly $n - n_1 - n_3 = n' + n_2 + n_4$ parts remain (call them $B_1, \dots, B_{n' + n_2 + n_4}$), and we still need to account for n_2 loss events of type ℓ_2 , and n_4 loss events of type ℓ_4 .

Let L (for *large*) denote that set of all goods of value at least $(1 - \rho)MMS_i$ that are not in $\mathcal{S}_i$. Of them, let L' denote those that still remain available (neither allocated nor sold) when n' agents remain. At this stage of our analysis, they belong to the $n' + n_2 + n_4$ MMS_i -bundles that we have. We wish now to redistribute L' only among n' bundles, $B'_1, \dots, B'_{n'}$, those that will form our canonical ρ - MMS_i partition into n' parts. (We are allowed to also discard some goods from L' , as canonical partitions need not contain all goods.)

Consider first the case that $|L'| \leq n' + 1$. Observe that for $\rho = \frac{2}{3}$, no loss event causes a loss larger than MMS_i (type ℓ_2 loss is at most $\rho \cdot MMS_i$ and type ℓ_4 loss is at most $(3\rho - 1)MMS_i$). Hence the total value that we have at our disposal is at least $n' \cdot MMS_i$.

Proposition 6. *If $|L'| \leq n' + 1$, $\rho = \frac{2}{3}$, and the total value that we have at our disposal is at least $n' \cdot MMS_i$, then a canonical ρ - MMS_i partition into n' parts exists.*

Proof. The total value that we have at our disposal is composed of four disjoint components:

1. Goods in L' (*large* goods).

2. Money P' leftover from previously sold goods.
3. Goods in C' (here C stands for *cheap*) that were not sold until now, and are neither in $\mathcal{S}_i$ nor in L' . This last condition implies that $\bar{v}_i(e) \leq (1 - \rho)MMS_i$ for every $e \in C'$.
4. Goods in $\mathcal{S}'_i$ that were not sold until now and are in $\mathcal{S}_i$. We move some of the goods of $\mathcal{S}'_i$ into some of the other components (in particular, i gives up the intention to sell them), as follows.
 - (a) Those goods $e \in \mathcal{S}'_i$ with $\bar{v}_i(e) \leq (1 - \rho)MMS_i$ are moved into C' (in terms of their value they fit in C').
 - (b) If $|L'| < n'$, then $n' - |L'|$ goods $e \in \mathcal{S}'_i$ with $\bar{v}_i(e) > (1 - \rho)MMS_i$ (or fewer, if the number of available such goods is smaller) are moved into L' (in terms of their value they fit in L'). We reinterpret the notation $\mathcal{S}'_i$ to include only those goods not moved out of $\mathcal{S}'_i$. Observe that now every good $e \in \mathcal{S}'_i$ satisfies $\bar{v}_i(e) > (1 - \rho)MMS_i$, and moreover, if $\mathcal{S}'_i$ is non-empty then $|L'| \geq n'$.

If $|L'| \leq n'$, then put each good of L' in a different part of the canonical partition. If $|L'| = n' + 1$, then put two goods of L' in the same bundle B' , which can serve as one pure bundle of the canonical partition. In this latter case, we need to create only $n' - 1$ additional parts. The total value that remains at our disposal for this is at least $n' \cdot MMS_i - \bar{v}_i(B') \geq (n' - 2\rho)MMS_i$. To unify the cases of $|L'| \leq n'$ and $|L'| = n' + 1$, we rename $n' - 1$ as n' in the latter case, and so in either case we are in the situation that every good of L' is in a different part, and the total value at our disposal is at least $(n' + 1 - 2\rho)MMS_i$.

For money from P' , do a *bag filling procedure*, pouring money into the same part until the value of the part reaches $\rho \cdot MMS_i$, and then moving to the next part. Each part that is made acceptable in this manner is pure and has value at most $\rho \cdot MMS_i$.

For goods in C' , continue the bag filling procedure from the part in which it previously ended, putting them one by one into the same part until the value of the part reaches $\rho \cdot MMS_i$, and then moving to the next part. Each part that is made acceptable in this manner is pure and has value at most MMS_i .

At this stage we already have some parts that are acceptable and pure, some parts that contain only a single good from L' , and (at most) one part that is *special* (a part to which the bag filling procedure added goods but it has not become acceptable yet). Observe that no part can be empty. Having a part with no good from L' implies that $\mathcal{S}'_i$ is empty, but then the total combined value of all goods is at most $n' - 1$, whereas the total value at our disposal was higher, at least $n' + 1 - 2\rho$.

We now continue with the bundles that were not involved in the bag-filling, and are not acceptable in current form (where they contain a single item from L'). Let us denote the set of all such bundles J and order them arbitrarily from $j = 1$ to $j = |J|$. We iterate over the remaining goods in $\mathcal{S}'_i$ in an arbitrary order. For every such good e , we add it to bundle j in full if the current value of bundle j together with $p(e)$ does not exceed MMS_i . If we indeed add e , then this creates an acceptable bundle (its value is at least $2(1 - \rho)MMS_i \geq \rho \cdot MMS_i$, where the inequality holds for $\rho = \frac{2}{3}$). This bundle is pure and can be incorporated in the canonical partition. Otherwise, we sell e , and add exactly the amount of sale proceeds so that the value of bundle j is $\rho \cdot MMS_i$. We know that the remaining sale proceeds from e are of value at least $(1 - \rho)MMS_i$, or it could have been fully added to bundle j . We increase j by 1 and move to the next bundle, and we know that it can be made acceptable using the remaining proceeds from e , since the value of the good from L' is at least $(1 - \rho)MMS_i$, the remaining proceeds are at least $(1 - \rho)MMS_i$, and together this gives $2(1 - \rho)MMS_i = \rho MMS_i$ with $\rho = \frac{2}{3}$. We then repeat the logic we applied to bundle j repeatedly (either add the remaining sale proceeds in full if it does not exceed MMS_i , or exactly the amount of sale proceeds to reach ρMMS_i otherwise), and we are guaranteed that all bundles processed in this way will be acceptable and not exceed MMS_i , until all sale proceeds from e are

used. We then move on to the next good in $\mathcal{S}'_i$, and proceed in the same fashion. If all bundles in J are processed, then we add any remaining sale proceeds and goods from $\mathcal{S}'_i$ to the leftover bundle we marked during the bag-filling process.

We claim that when the process ends, we have a canonical partition. This is because we have three type of bundles: pure bundles (which can be part of the canonical partition), one leftovers bundle (this is allowed in a canonical partition), and bundles that contain only a single full good (this good is from L') and sale proceeds from a single good. Importantly, the process where we add proceeds from $\mathcal{S}'_i$ to bundles in J suffices in order to make all such bundles acceptable, and the leftover bundle as well. That is because we maintain that no bundle has value above MMS_i . Since the total value at our disposal is at least $(n' + 1 - 2\rho)MMS_i = (n' - 1 + \rho)MMS_i$ (the last inequality holds for $\rho = \frac{2}{3}$). I.e., we have enough value to complete the bundle processing creating the $n' - 1$ bundles other than the leftover bundle using at most $(n' - 1)MMS_i$ of value, and with at least $\rho \cdot MMS_i$ value left for the leftover bundle, making it acceptable.

Given Proposition 6, it remains to handle the case that $|L'| > n' + 1$.

We do this in a sequence of transformations that reduce it to the case of $|L'| \leq n' + 1$. To present this sequence of transformations, we introduce some notation.

Recall that there are n' active agents, and there are n_2 loss events of type ℓ_2 and n_4 loss events of type ℓ_4 . We have a partition into $n' + n_2 + n_4$ bundles (each bundle may contain goods, and also money from goods previously sold, and money from goods that i intends to sell), where each has $\bar{v}_i$ value exactly MMS_i . (If there were parts of higher value, we may reduce the values of some of the goods or give up some of the payments so that each part has value exactly MMS_i , see Corollary 6). In our transformations we shall create some acceptable pure bundles. Hence, the effective value of n' will go down, because fewer additional bundles need to be created for the partition. We let n_0 denote the number of additional bundles that need to be created. Likewise, in the process we may discard goods that were involved in loss events, and consequently also n_2 and n_4 might change their values. We refer to their new values as n'_2 and n'_4 . Finally, also the set L' may shrink (as some of its goods may become part of pure bundles that we created). We refer to the set of goods remaining in L' as L_0 . Our goal is to reach a stage in which $|L_0| \leq n_0 + 1$, while maintaining that the total value that we have at our disposal is at least $n_0 \cdot MMS_i$. This will allow us to apply Proposition 6.

Recall that each loss event of type ℓ_4 involves two goods, e that is part of the allocated bundle, and f that is sold and some of the money received is part of the allocated bundle, and the loss is at most $v_i(f) + (2\rho - 1)MMS_i$. Below, whenever we name a good f_j (for some j), this signifies that the good participated as the f good in a loss event of type ℓ_4 .

Removing redundant loss events. Suppose that the current partition has a bundle B_j that contains two goods, f_1 and f_2 , from loss events of type ℓ_4 . The two loss events of type ℓ_4 contributed a loss of at most $v_i(f_1) + (2\rho - 1)MMS_i + v_i(f_2) + (2\rho - 1)MMS_i \leq \bar{v}_i(B_j) + (4\rho - 2)MMS_i \leq \bar{v}_i(B_j) + \rho \cdot MMS_i$ (the last inequality holds for our choice of $\rho = \frac{2}{3}$). As B_j is part of the current partition we have that $\bar{v}_i(B_j) \leq MMS_i$. Replace the two ℓ_4 loss events by a loss event for the remaining $\rho \cdot MMS_i$, which qualifies as a loss event of type ℓ_2 , and remove B_j .

The number of bundles in the partition decreased by one, n'_4 decreased by two, whereas n'_2 increased by one.

As the transformation above reduces the number of bundles, it can be employed only finitely many times. Hence, we necessarily reach a stage in which no bundle B_j contains two goods of type f from loss events of type ℓ_4 .

Handling pairs of goods from L' in the same bundle. Suppose that the current partition has a bundle B_j that contains two complete goods from L' . Then we create from it a pure bundle B' in the canonical partition by including only these two goods, and discarding whatever is left

from B_j (either goods or money). We have that $\bar{v}_i(B') \geq 2(1 - \rho)MMS_i = \rho \cdot MMS_i$, where the last equality holds because $\rho = \frac{2}{3}$. Without B' , the total number of bundles decreased by one, n_0 decreased by one, $|L'_0|$ decreased by two, and n'_2 and n'_4 remain unchanged.

Necessarily, we reach a stage in which every bundle B_j contains at most one good from L' .

Mixed L' and f bundles. Suppose that the current partition has two bundles, call them B_1 and B_2 , each containing one good (f_1 and f_2 , respectively), where each of these goods is in a loss event of type ℓ_4 . In addition, B_1 and B_2 , each contain one good (d_1 and d_2 , respectively) from L' . Each B_j may possibly contain additional goods and money. Then create the acceptable pure bundle $B' = \{d_1, d_2\}$ and put it in the canonical partition, and discard the remaining parts of B_1 and B_2 (thus discarding also f_1 and f_2). The number of remaining bundles decreased by two, n_0 decreased by one, n'_4 decreased by 2 and n'_2 increased by 1 (the claims regarding n'_4 and n'_2 are as in *removing redundant loss events*).

Necessarily, we reach a stage in which at most one bundle B_j contains both an f good from an ℓ_4 loss event and a good from L' .

At this stage we have n'_2 loss events of type ℓ_2 , n'_4 loss events of type ℓ_4 , we need to create n_0 additional bundles for the canonical partition, and we have $n_0 + n'_2 + n'_4$ bundles, each of value MMS_i . Of these, n'_4 bundles contain a good f from an ℓ_4 loss event, at most one of which contains a good from L_0 . Moreover, all goods from L_0 are in separate bundles. Hence, $|L_0| \leq n_0 + n'_2 + 1$. If $|L_0| \geq 2n_0$ then we can create n_0 pairs of goods from L_0 . Each such pair makes an acceptable pure bundle, and so we are done. Hence we may assume that $|L_0| < 2n_0$. Likewise, we may assume that $|L_0| > n_0 + 1$, as otherwise we can apply Proposition 6.

Create $t = |L_0| - n_0 - 1 \leq n'_2$ acceptable pure bundles, each containing two goods from L_0 . Notice that this is possible as $\frac{1}{2}|L_0| \leq n_0 + 1$, and so $t = |L_0| - n_0 - 1 \leq \frac{1}{2}|L_0|$. We remark that each of the pure bundles that we create in this step might have value as high as $2\rho \cdot MMS_i > MMS_i$. Thus, it might appear as if the value that remains at our disposal drops below MMS_i times the number of bundles that we still need to create. However, the next paragraph shows the latter does not happen.

Afterwards, the number of bundles still needed for the canonical partition is $\hat{n} = n_0 - t$, whereas the new size of L_0 equals to $\hat{L} = |L_0| - 2t$. However, we have $\hat{L} = \hat{n} + 1$, as $|L_0| - 2t = t + n_0 + 1 - 2t = n_0 - t + 1 = \hat{n} + 1$. The total value left at our disposal, which we denote by G , is at least $G \geq (n_0 + n'_2 + n'_4)MMS_i - n'_2\rho MMS_i - n'_4 \cdot MMS_i - 2t\rho MMS_i$ (since each pure bundle created has value at most 2ρ). As $n'_2 \geq t$, we have that $G \geq (n_0 + (1 - 3\rho)t)MMS_i = (n_0 - t)MMS_i = \hat{n}MMS_i$ (for $\rho = \frac{2}{3}$). Thus, we need to construct $\hat{n}$ additional bundles in the canonical partition, have at most $\hat{n} + 1$ large goods, and at least $\hat{n} \cdot MMS_i$ total value for our disposal. Proposition 6 thus applies, and we obtain the canonical partition.

Lemma 9 established that when $\rho = \frac{2}{3}$, canonical partitions can be found whenever needed by the allocation algorithm. Eventually, every agent receives a bundle that is acceptable to her. This proves the general n part of Theorem 3.

ALGORITHM 2: FIND-CANONICAL

Input: Valuations v over goods $\mathcal{M}$, market-price p , number of active agents n' , $B_1, \dots, B_n$ an MMS partition according to v , Loss events $\ell_1, \dots, \ell_4$

Output: A canonical partition $B_1, \dots, B_{n'}$ and $\forall 1 \leq j \leq n', \bar{v}(B_j) \geq \frac{2}{3} \cdot MMS$

```

1  if  $|\ell_1| > 0$  then
2    Arrange the bundles so that bundles corresponding to a loss event in  $\ell_1$  have higher indices.
3    return FIND-CANONICAL( $v, p, n', B_1, \dots, B_{n-|\ell_1|}, \emptyset, \ell_2, \ell_3, \ell_4$ ).
4  end
5  if  $|\ell_3| > 0$  then
6    Let  $e, f$  be the goods associated with  $\ell_3$ , and  $r$  the payment associated with it.
7    if  $\exists j, \{e, f\} \subseteq B_j$  then
8      Remove  $B_j$ 
9    end
10   else
11     Let  $j_1, j_2$  be the bundles containing  $e, f$  respectively. Let  $B' = (B_{j_1} \cup B_{j_2} \setminus \{e, f\}) \cup r$ .
12     Remove  $B_{j_1}, B_{j_2}$ , and add  $B'$  instead.
13   end
14   return FIND-CANONICAL( $v, p, n', B, \ell_1, \ell_2, \ell_3 \setminus \ell_3, \ell_4$ ).
15 end
16 Let  $L'$  be goods of value  $\bar{v}(g) \geq (1 - \rho)MMS$  not in  $\mathcal{S}$ .
17 if  $|L'| \leq n' + 1$  then
18   Let  $P'$  be money from goods that do not appear in full in  $B_1, \dots, B_n$  (i.e., already sold in previous loss
19   events). Let  $\mathcal{S}'$  be the goods newly sold in the MMS partition. Let  $C'$  be goods not in  $\mathcal{S} \cup L'$ .
20   return UNSTRUCTURED-FIND-CANONICAL( $v, p, n', L', P', C', \mathcal{S}'$ ).
21 end
22 if  $\exists j, l_1 = (e_1, f_1) \in \ell_4, l_2 = (e_2, f_2) \in \ell_4, \{f_1, f_2\} \subseteq B_j$  then
23   Remove  $B_j$ . Create a loss event  $l = \{e_1, e_2\}$  of type  $\ell_2$ .
24   return FIND-CANONICAL( $v, p, n', B, \ell_1, \ell_2 \cup \{l\}, \ell_3, \ell_4 \setminus \{l_1, l_2\}$ )
25 end
26 if  $\exists j, |L' \cap B_j| \geq 2$  then
27   Let  $B' = L' \cap B_j$ .
28   return  $B'$ , FIND-CANONICAL( $v, p, n' - 1, B \setminus B_j, \ell_1, \ell_2, \ell_3, \ell_4$ ).
29 end
30 if  $\exists j_1, j_2, B_{j_1} \cap L' = \{d_1\}, B_{j_2} \cap L' = \{d_2\}, l_1 = (e_1, f_1) \in \ell_4, f_1 \in B_{j_1}, l_2 = (e_2, f_2) \in \ell_4, f_2 \in B_{j_2}$  then
31   Let  $B' = \{d_1, d_2\}$ . Create a loss event  $l$  with  $l = \{e_1, e_2\}$  of type  $\ell_2$ .
32   return  $B'$ , FIND-CANONICAL( $v, p, n' - 1, B \setminus (B_{j_1} \cup B_{j_2}), \ell_1, \ell_2 \cup \{l\}, \ell_3, \ell_4 \setminus \{l_1, l_2\}$ ).
33 end
34 if  $|L'| \geq 2n'$  then
35   Create  $A_1, \dots, A_{n'}$  bundles with pairs of goods from  $L'$ .
36   return  $A_1, \dots, A_{n'}$ .
37 end
38 else
39   Let  $t = |L'| - n' - 1$ . Create  $A_1, \dots, A_t$  bundles with pairs of goods from  $L'$ . Remove all goods in  $\cup_{j=1}^t A_j$ 
40   from  $L'$ .
41   Let  $P'$  be money from goods that do not appear in full in  $B_1, \dots, B_n$  (i.e., already sold in previous loss
42   events). Let  $\mathcal{S}'$  be the goods newly sold in the MMS partition. Let  $C'$  be goods not in  $\mathcal{S} \cup L'$ .
43   return  $A_1, \dots, A_t$ , UNSTRUCTURED-FIND-CANONICAL( $v, p, n' - t, L', P', C', \mathcal{S}'$ ).
44 end

```

ALGORITHM 3: UNSTRUCTURED-FIND-CANONICAL

Input: Valuations v over goods $\mathcal{M}$, market-price p , number of active agents n' , L' large goods, P' sale proceeds, C' small goods, S' unsold large goods

Output: A canonical partition $B_1, \dots, B_{n'}$ and $\forall 1 \leq j \leq n', \bar{v}(B_j) \geq \frac{2}{3} \cdot MMS$

- 1 Put a good of L' in each bundle $A_1, \dots, A_{n'}$. If a good remains, add it to A_1 .
- 2 Bag-fill bundles using proceeds from P' up to size ρMMS .
- 3 Bag-fill bundles using goods in C' , with a threshold of ρMMS .
- 4 Bag-fill bundles that still contain a single good of L' , each with a single good from S' , and without exceeding MMS . If adding a good exceeds MMS , sell it, add exactly the amount of proceeds to reach ρMMS , and treat the remaining proceeds as a virtual good.
- 5 return $A_1, \dots, A_{n'}$.

B.1 Achieving $\frac{3}{4}$ -MMS with $n = 3$

Lemma 3. *Given an MMS partition for $n = 3$, and where any good g has $\bar{v}_i(g) < \frac{3}{4} \cdot MMS_i$, we can create a canonical form partition of acceptable bundles each of value at least $\frac{3}{4} \cdot MMS_i$.*

Proof. By Lemma 5, we can consider an MMS partition that has at most 2 goods that are sold. If the MMS partition has no sold goods, then all bundles are pure, and it is a canonical partition as is. We thus assume there is at least one sold good.

If at most one bundle in the MMS output has more than $\frac{1}{4} \cdot MMS$ in sale proceeds, we can declare it as the leftovers bundle, remove any sale proceeds from the other two bundles, and declare them as pure bundles. This gives a canonical partition where each bundle has value at least $\frac{3}{4} \cdot MMS$. Thus, there must be at least two such bundles, and overall $\frac{1}{2} \cdot MMS$ in sale proceeds. Since, as we showed, we can assume between one and two sold goods, we must have a good with at last $\frac{1}{4} \cdot MMS$ in sale proceeds. Denote it F .

Next, we consider the case where we can create a singleton bundle with sale proceeds from F and a full good e , where the bundle's value is exactly $\frac{3}{4} \cdot MMS_i$. In this case, we let B_1 be the singleton bundle. We do a bag-filling procedure to create B_2 , with the threshold at $\frac{3}{4} \cdot MMS_i$. Since for each good g , $\bar{v}_i(g) \leq \frac{3}{4} \cdot MMS_i$, B_2 has value at most $1.5 \cdot MMS_i$. Overall, the remaining goods have value of at least $(3 - (\frac{3}{4} + 1.5)) \cdot MMS_i = \frac{3}{4} \cdot MMS_i$. This is a partition with a value of at least $\frac{3}{4} \cdot MMS_i$ for each bundle, and it is canonical, as it is made of a singleton, pure, and a single leftovers bundle.

If we cannot create such a singleton bundle, this implies that all goods $g \in \mathcal{M} \setminus \{F\}$ have $\bar{v}_i(g) < \frac{1}{2} \cdot MMS_i$, or otherwise, we could create a singleton bundle made of g and sale proceeds from F .

We consider the following construction: We use *greedy* bag-filling using goods in $\mathcal{M} \setminus \{F\}$ to create B_1, B_2 with a threshold of $\frac{3}{4} \cdot MMS_i$, and put all remaining goods (including F) in B_3 . By greedy bag-filling we mean adding goods in order of their $\bar{v}_i$ value (larger goods earlier), and moving to the next bundle once the current bundle has a value of at least $\frac{3}{4} \cdot MMS_i$. This is a canonical partition as B_1, B_2 are pure, and B_3 is a single leftovers bundle. So, it remains to establish that $\bar{v}_i(B_3) \geq \frac{3}{4} MMS_i$.

Since we established that for all goods $g \in \mathcal{M} \setminus \{F\}$ have $\bar{v}_i(g) < \frac{1}{2}$, it must be that B_1 contains at least two goods. First, consider if it contains exactly two goods. Then, its total $\bar{v}_i$ value is at most MMS_i . The value of B_2 is at most $(\frac{3}{4} + \frac{1}{2}) MMS_i = \frac{5}{4} MMS_i$, since the last good added to the bag-filling is of value at most $\frac{1}{2} MMS_i$, and the value of the bundle before adding it is below $\frac{3}{4} MMS_i$. Overall, we conclude that B_3 has value of at least $(3 - (1 + \frac{5}{4})) \cdot MMS_i = \frac{3}{4} MMS_i$.

Lastly, consider if B_1 contains at least 3 goods. Then, the two larger goods in B_1 have value less than $\frac{3}{4} \cdot MMS_i$, and so the second largest good must have value of less than $\frac{3}{8} \cdot MMS_i$. All subsequently added goods (both to B_1 and B_2) have value of at most $\frac{3}{8} \cdot MMS_i$. We thus have that

both B_1 and B_2 have a value of at most $(\frac{3}{4} + \frac{3}{8}) \cdot MMS_i = \frac{9}{8} \cdot MMS_i$, which guarantees a value of at least $(3 - 2 \cdot \frac{9}{8}) \cdot MMS_i = \frac{3}{4} \cdot MMS_i$ for B_3 .

Lemma 4. *Suppose that B is either a singleton bundle or a pure bundle, and $\bar{v}_i(B) \leq \frac{3}{4}MMS_i$ holds for agent i . Then if B is allocated to some other agent, agent i can partition the remaining goods into two disjoint bundles B'_1, B'_2 , each of value $\bar{v}_i(B'_j) \geq \frac{3}{4}MMS_i$.*

Proof. Consider an MMS output for agent i be B_1, B_2, B_3 , each with value MMS_i . A bundle given away to some agent by the algorithm in stages (i)-(iii), results in a loss event for agent i of one of the four types specified in Definition 7. We specify the construction of B'_1, B'_2 depending on the loss event type. Notice that for the purpose of this lemma, our construction does not need to be canonical.

- ℓ_1 . With this loss event, a single good is eliminated, which the agent does not sell (but rather keeps) in the MMS partition. It thus must belong to a single bundle, say B_1 . Then, we can set $B'_1 = B_2, B'_2 = B_3$ to serve as the two disjoint bundles required by the lemma.
- ℓ_2 . With this loss event, the value lost is at most $\frac{3}{4}MMS_i$. We do greedy bag-filling into bundles B'_1, B'_2 with a threshold of $\frac{3}{4}$ -MMS, where if the good that passes the threshold is meant to be sold in the MMS partition, we only include sale proceeds to arrive at a value of $\frac{3}{4}$ -MMS. Consider if B'_1 contains a single good. Then its value is at most MMS (if it is a good meant to be kept, it has at most a value of MMS, and if it is meant to be sold, then we only take $\frac{3}{4}$ -MMS of sale proceeds). We then have a remaining value of 1.5-MMS (after removing the ℓ_2 loss event and B'_1), and so B'_2 must be successfully filled. If B'_1 contains at least two goods, then the first added good has a value of at most $\frac{3}{4}$ -MMS, and since this is greedy bag-filling, this means that any subsequently added good has a value of at most $\frac{3}{4}$ -MMS as well. Overall, this means B'_1 has a value of at most $(\frac{3}{4} + \frac{3}{4}) \cdot MMS = 1.5 \cdot MMS$, and we have a remaining value of at least $\frac{3}{4}$ -MMS (after removing the ℓ_2 loss event and B'_1), and so B'_2 must be successfully filled.
- ℓ_3 or ℓ_4 . With either of these loss types, we have two goods e, f , where f is not sold as part of the MMS partition, and a maximal combined loss of $v_i(f) + (2\rho - 1)MMS_i \leq (\frac{3}{4} + \frac{1}{2})MMS_i = \frac{5}{4}MMS_i$. We make two observations: (i) f is in a single bundle of B_1, B_2, B_3 (since it is not sold in the MMS partition), say B_1 . We can remove f in full from B_1 , and add back unused sale proceeds (as f is used in the loss event for its sale proceeds) as we wish to the other bundles. By the combined loss bound, we can ensure that one of the two other bundles has a loss of at most $\frac{1}{4} \cdot MMS$ by removing e (in full if it is kept in the MMS partition, or its various sale proceeds if it is sold), and adding back unused sale proceeds of f as necessary. This is because the combined loss bound guarantees other than $v_i(f)$ we lose at most $\frac{1}{2} \cdot MMS_i$. W.l.o.g., this means that the revised bundle B_2 (denote it B'_2 after the changes) has a value of at least $\frac{3}{4} \cdot MMS$. (ii) Gather all remaining goods and sale proceeds in a bundle B'_1 . In the loss event, at most $\frac{5}{4}MMS_i$ were removed, and B'_2 has at most value of MMS_i (as it was created out of B_2 , where we only add sale proceeds from f if necessary to complete its value to $\frac{3}{4}MMS_i$, but anyway not exceeding the original value of MMS_i). This leaves at least $(3 - (\frac{5}{4} + 1))MMS_i = \frac{3}{4}MMS_i$ in B'_1 , and so this is an acceptable partition.

Given that each agent i of the two remaining agents can partition the remaining items to two disjoint bundles with value $\frac{3}{4} \cdot MMS_i$, we can apply Theorem 1 with the remaining items and two remaining agents, and this guarantees each of them $\frac{3}{4}MMS_i$.

C Existence Results for Envy-Free Notions

We follow our envy notions definitions of Definition 4. We distinguish between settings in which all goods are sellable, and those in which some goods are sellable and some are not.

If all goods are sellable, then an EF allocation exists: sell all goods and divide the money equally. Even if not all goods are sellable, in essence, the EFX existence and SEFX existence situation are similar:

Lemma 10. *If for some number of agents n and additive valuations, an EFX allocation always exists, then the same is true for SEFX, and vice versa. The same holds for EF1 and SEF1.*

Proof. We prove the lemma for EFX and SEFX, and the proof for EF1 and SEF1 is the same.

The SEFX $\implies$ EFX implication is straight-forward as remarked above. In the other direction, [9] show that for a mix of divisible and indivisible goods, if an EFX allocation exists for the indivisible goods, it can be extended to an EFX allocation of divisible and indivisible goods, by only adding divisible cake to the existing allocations. Thus, if an EFX allocation is guaranteed to exist for some n , we can find an EFX allocation for all the goods that are not sellable, sell all the sellable goods, and extend the EFX allocation to an EFX allocation of divisible and indivisible goods using the sale proceeds from the sellable goods. Any indivisible good g allocated in this setting has $p(g) = 0$ (since it is not sellable), and so the SEFX condition is the same as that of an EFX allocation of divisible and indivisible goods.

However, an allocation that only satisfies envy-freeness properties might have very little value for the agents. Hence, we seek allocations that satisfy a relaxed version of EF, but also guarantee good value to each agent.

As a useful technical step, we now define ϵ notions of envy, but our interest is in the case when $\epsilon = 0$. We consider the more general definition since later in the discussion, it helps to consider the ϵ notions (with the end-goal of proving existence for the $\epsilon = 0$ case).

Definition 8. *An allocation $(A_1, P_1), \dots, (A_n, P_n)$ is envy free (EF) if for every agent i and other agent j , $\bar{v}_i(A_i) + P_i \geq \bar{v}_i(A_j) + P_j$. An allocation satisfies the following relaxations of EF if the above envy free condition holds whenever $P_j > 0$, and either the EF condition or the following conditions hold if $P_j = 0$.*

1. ϵ -SEF1 (strong/sellable envy free up to one good). $\bar{v}_i(A_i) + P_i + \epsilon \geq \bar{v}_i(A_j \setminus \{g\}) + p(g)$ for some good $g \in A_j$.
2. ϵ -SEFX (strong/sellable envy free up to any good). $\bar{v}_i(A_i) + P_i + \epsilon \geq \bar{v}_i(A_j \setminus \{g\}) + p(g)$ for every good $g \in A_j$.

The following lemma is technically useful in simplifying some of the proofs. Due to the existence of divisible sales-proceeds, envy might be by an infinitesimally small amount. By considering ϵ -SEFX rather than SEFX, iterative algorithms that we design make finite progress in each step rather than infinitesimally small progress, allowing us to conclude that they terminate in finite time.

Lemma 11. *For a given allocation instance with sellable goods and a given value of ρ and individual share values s_i , suppose that for every $\epsilon > 0$ there is an allocation (a partial allocation, respectively) that is $(\rho - \epsilon) \cdot s_i$ and ϵ -SEFX. Then there also is an allocation (partial allocation, respectively) that is $\rho \cdot s_i$ and SEFX.*

Proof. We prove the lemma for partial allocations and SEFX.

In any partial allocation, each good is either allocated to a single agent, or sold, or not allocated at all. Hence, there are $(n+2)^m$ classes of partial allocations, conditioned on the decision for each good. Within a class, one need only distribute the money so as to minimize ϵ . Refer to the minimum attainable value of ϵ within a class c as ϵ_c . Minimizing ϵ_c (subject to $\epsilon_c \geq 0$) is a solution to a linear program (as both the share guarantee and SEFX constraints are linear in the sale proceeds and ϵ_c) over a closed and bounded set. As such, the optimal value of ϵ_c is either 0 or bounded away from 0. Consequently, the optimal value of ϵ (which is the minimum over ϵ_c for finitely many indices c) is either 0 or bounded away from 0. Under the terms of the lemma, it must be 0.

D Algorithms and Missing Proofs for TPS Approximation

Let $f(t) = \frac{1}{n} \sum_{j \in \mathcal{M}} \max\{p(j), \min\{v(j), t\}\}$.

The following claims is useful:

Claim. If for some t , $f(t) \geq t$, then $TPS(\mathcal{M}, v, p, n) \geq t$.

Proof. If $f(t) = t$, then $TPS(\mathcal{M}, v, p, n) \geq t$ holds by the definition of TPS as the maximal t value satisfying this equality. If $f(t) > t$, notice that $g(x) = f(x) - x$ is a continuous function with $g(t) > 0$.

By Lemma 1, $g(t') < 0$ for any $t' > PS(\mathcal{M}, v, p, n)$. Thus, there must be some t' so that $t' > t$ and $g(t') < 0$. By the median value theorem, there is some $t < t'' < t'$ with $g(t'') = 0$, and so $TPS(\mathcal{M}, v, p, n) \geq t'' > t$.

Lemma 1. $MMS(\mathcal{M}, v, p, n) \leq TPS(\mathcal{M}, v, p, n) \leq PS(\mathcal{M}, v, p, n)$.

Proof. ($TPS \leq PS$)

Let $t' > PS(\mathcal{M}, v, p, n)$. Then,

$$f(t') = \frac{1}{n} \sum_{j \in \mathcal{M}} \max\{p(j), \min\{v(j), t'\}\} \leq \frac{1}{n} \sum_{j \in \mathcal{M}} \max\{p(j), v(j)\} < t'.$$

Since the TPS is the maximal value so that the above holds as equality, the TPS must be at most the proportional share.

($MMS \leq TPS$)

Consider an MMS partition $C_1, \dots, C_n$. Let $\mathcal{S}_C$ be the goods sold in the partition, and V_C the goods kept. Every good kept j contributes at most

$$\min\{v(j), MMS\}$$

to their bundle (and they are included in a single bundle), and every sold good contributes at most $p(j)$ to all bundles. If we sum over the n MMS bundles, we get $n \cdot MMS \leq \sum_{j \in \mathcal{S}_C} p(j) + \sum_{j \in V_C} \min\{v(j), MMS\}$, and thus (dividing by n) we get:

$$\begin{aligned} MMS &\leq \frac{1}{n} \left(\sum_{j \in \mathcal{S}_C} p(j) + \sum_{j \in V_C} \min\{v(j), MMS\} \right) \\ &\leq \frac{1}{n} \sum_{j \in \mathcal{S}_C} \max\{p(j), \min\{v(j), MMS\}\} \end{aligned}$$

It then follows that $TPS(\mathcal{M}, v, p, n) \geq MMS$ by Claim D.

The TPS is efficiently computable, by the following simple algorithm.

Consider the set of (at most $2m + 1$) points $S = \{0\} \cup \bigcup_{j \in \mathcal{M}} \{v(j), p(j)\}$, and sort the points in decreasing order $s_1, \dots, s_{|S|}$. Find the first point k so that $f(s_k) \geq s_k$.

Claim. If $k = 1$, then $TPS(\mathcal{M}, v, p, n) = PS(\mathcal{M}, v, p, n)$. If $k > 1$, then $s_k \leq TPS(\mathcal{M}, v, p, n) < s_{k-1}$.

Proof. By Claim D, the TPS is at least s_k .

If $k = 1$, then for $t = s_1$, it holds that $t \geq v(j)$ for all $j \in \mathcal{M}$. Thus, $f(s_1) = \frac{1}{n} \sum_{j \in \mathcal{M}} \bar{v}(j)$, and f is weakly monotonically increasing in t . Thus, $f(PS(\mathcal{M}, v, p, n)) \geq f(s_1) = PS(\mathcal{M}, v, p, n)$, and so

$$PS(\mathcal{M}, v, p, n) \stackrel{\text{Claim D}}{\leq} TPS(\mathcal{M}, v, p, n) \stackrel{\text{Lemma 1}}{\leq} PS(\mathcal{M}, v, p, n).$$

If $k > 1$, then for any $k' < k$, $f(s_{k'}) < s_{k'}$. Thus, none of the points can be the TPS (as it does not satisfy the equality condition), but also the interior intervals cannot be the TPS, as within each interval f is linear since for each j , $\max\{p(j), \min\{v(j), t\}\}$ remains either equal to $p(j)$, $v(j)$, or t , and f is the sum of these linear expressions. If there was an interior point so that $f(t) = t$, and given $f(s_{k'}) < s_{k'}$, that implies that $f(t) - t$ is increasing within the interval, and so it must be that $f(s_{k'+1}) \geq s_{k'+1}$, in contradiction.

Thus, if $k = 1$ we can directly compute the TPS by Claim D, and if $k > 1$, we can pinpoint the interval where the TPS resides. Since f takes a linear form within the interval $[s_k, s_{k-1})$, we can solve the one-variale linear equation $f(t) = t$ with the appropriate coefficients, which we can determine by choosing a point $t \in (s_k, s_{k-1})$, and seeing whether $\max\{p(j), \min\{v(j), t\}\}$ equals $p(j)$, $v(j)$, or t , for every $j \in \mathcal{M}$.

D.1 TPS Approximation

ALGORITHM 4: APX-TPS

Input: Valuation functions $v_1, \dots, v_n$ and market price function p over goods $\mathcal{M}$, number of agents n

Output: An allocation A, P with at least $\frac{n}{2n-1} \cdot TPS$ of value for each agent

```

1  $AVAIL = \mathcal{M}$ 
2  $ACTIVE = [n]$ 
3 while  $\exists j \in AVAIL, i \in ACTIVE, p(j) \geq \frac{n}{2n-1} TPS_i$  do
4   | Let  $i^* \in \arg \min_{i \in ACTIVE} TPS_i$ .
5   | Assign  $\frac{n}{2n-1} TPS_{i^*}$  in sale-proceeds from selling  $j$  to agent  $i^*$ .
6   |  $D'_j = p(j) - \frac{n}{2n-1} TPS_{i^*}$  // The remaining sale proceeds from  $j$ , which we treat as a virtual good.
7   |  $AVAIL = AVAIL \setminus \{j\} \cup \{D'_j\}, ACTIVE = ACTIVE \setminus \{i^*\}$ .
8 end
9 while  $\exists j \in AVAIL, i \in ACTIVE, v_i(j) \geq \frac{n}{2n-1} TPS_i$  do
10  | Assign  $j$  to agent  $i$ .
11  |  $AVAIL = AVAIL \setminus \{j\}, ACTIVE = ACTIVE \setminus \{i\}$ .
12 end
13 while  $|ACTIVE| \neq 0$  do
14  | Bag-fill complete goods into a bundle  $B$ , until some agent  $i \in ACTIVE$  values it at least  $\frac{n}{2n-1} TPS_i$ .
    | Assign  $B$  to agent  $i$  (if there are several agents, choose one arbitrarily).
15  |  $AVAIL = AVAIL \setminus B, ACTIVE = ACTIVE \setminus \{i\}$ .
16 end

```

Consider a good j that was sold and some of its sale proceeds were allocated. Let D'_j be the remaining sale proceeds. Then, we can treat D'_j as a new *virtual indivisible good*. The good has the value D'_j for all agents. We can “sell” it, which means taking some of its sale proceeds, or we can “allocate” it as a complete good (during bag-filling).

Lemma 2. *In the setting with indivisible sellable goods and additive valuations, the algorithm described above for TPS approximation (with pseudo-code in Algorithm 4) produces a partial allocation that is $\frac{n}{2n-1}$ -TPS.*

Proof. Any agent i that is allocated a bundle gets at least $\frac{n}{2n-1}TPS_i$. This is since it receives a bundle for the first time either in the large goods selling loop (line 3), the large good keeping loop (line 9), or in the bag-filling loop (line 13), whereas in each of these cases it has a value of at least $\frac{n}{2n-1}MMS_i$ for the bundle.

We show that every agent gets allocated a bundle during the run of the algorithm.

Lemma 12. *For every active agent i , after k agents are allocated a bundle, it values the remaining goods at least $(n - \frac{2nk}{2n-1})TPS_i$.*

Proof. Let K_1 be the agents allocated a bundle in the first loop out of goods g so that agent i has $p(g) \geq \frac{n}{2n-1}TPS_i$, K_2 agents allocated a bundle in the first loop out of goods agent i has $p(g) < \frac{n}{2n-1}TPS_i$, and K_3 agents allocated a bundle in the second loop. We have $k = \sum_{t=1}^3 |K_t|$.

Notice the following about goods in K_1 : It holds that $TPS_i \leq \frac{2n}{2n-1}TPS_i = \frac{n}{2n-1}TPS_i + p(g)$. Then,

$$\max\{p(g), \min\{v_i(g), TPS_i\}\} \leq \max\{p(g), TPS_i\} \leq p(g) + \frac{n}{2n-1}TPS_i + p(g). \quad (8)$$

Notice the following about goods in K_2 : If $p(g) > v_i(g)$, then

$$\begin{aligned} \max\{p(g), \min\{v_i(g), TPS_i\}\} &= \max\{p(g), TPS_i\} \\ &\leq \max\left\{\frac{n}{2n-1} \cdot TPS_i, TPS_i\right\} \leq TPS_i. \end{aligned}$$

If $p(g) \leq v_i(g)$, then $\max\{p(g), \min\{v_i(g), TPS_i\}\} = \min\{v_i(g), TPS_i\} \leq TPS_i$. To conclude, in both cases,

$$\max\{p(g), \min\{v_i(g), TPS_i\}\} \leq TPS_i. \quad (9)$$

We can write, by the definition of TPS_i :

$$\begin{aligned} &n \cdot TPS_i \\ &= \sum_{g \in \mathcal{M}} \max\{p(g), \min\{v_i(g), TPS_i\}\} \\ &\stackrel{\text{Eq. 9}}{\leq} \sum_{g \in \mathcal{M} \setminus \cup_{j \in K_2} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} + \sum_{g \in \cup_{j \in K_2} B_j} TPS_i \\ &\leq \sum_{g \in \mathcal{M} \setminus \cup_{j \in K_2} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} + |K_2|TPS_i \\ &\stackrel{\text{Eq. 8}}{\leq} \sum_{g \in \mathcal{M} \setminus \cup_{j \in (K_1 \cup K_2)} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} + |K_1| \cdot \frac{n}{2n-1}TPS_i \end{aligned} \quad (10)$$

$$\begin{aligned}
& + \sum_{g \in \cup_{j \in K_1} B_j} p(g) + |K_2| TPS_i \\
& = \sum_{g \in \mathcal{M} \setminus \cup_{j \in (K_1 \cup K_2)} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} \\
& + \sum_{g \in \cup_{j \in K_1} B_j} \left(p(g) - \frac{n}{2n-1} TPS_i \right) + |K_1| \frac{2n}{2n-1} TPS_i + |K_2| TPS_i \\
& = \sum_{g \in \mathcal{M} \setminus \cup_{j \in (K_1 \cup K_2 \cup K_3)} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} \tag{11}
\end{aligned}$$

$$\begin{aligned}
& + \sum_{j \in K_3} \sum_{g \in B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} \\
& + \sum_{g \in \cup_{j \in K_1} B_j} \left(p(g) - \frac{n}{2n-1} TPS_i \right) + |K_1| \frac{2n}{2n-1} TPS_i + |K_2| TPS_i \\
& = \sum_{g \in \mathcal{M} \setminus \cup_{j \in (K_1 \cup K_2 \cup K_3)} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} + \sum_{j \in K_3} \sum_{g \in B_j} \bar{v}_i(g) \\
& + \sum_{g \in \cup_{j \in K_1} B_j} \left(p(g) - \frac{n}{2n-1} TPS_i \right) + |K_1| \frac{2n}{2n-1} TPS_i + |K_2| TPS_i \\
& \leq \sum_{g \in \mathcal{M} \setminus \cup_{j \in (K_1 \cup K_2 \cup K_3)} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} + |K_3| \frac{2n}{2n-1} TPS_i \tag{12} \\
& + \sum_{g \in \cup_{j \in K_1} B_j} \left(p(g) - \frac{n}{2n-1} TPS_i \right) + |K_1| \frac{2n}{2n-1} TPS_i + |K_2| TPS_i.
\end{aligned}$$

Now, we can reduce terms from both sides, and arrive at:

$$\begin{aligned}
& \left(n - \frac{2n \cdot k}{2n-1} \right) \cdot TPS_i = \left(n - \frac{2n}{2n-1} (|K_1| + |K_2| + |K_3|) \right) \cdot TPS_i \\
& \leq \left(n - |K_2| - \frac{2n}{2n-1} |K_1| - \frac{2n}{2n-1} |K_3| \right) \cdot TPS_i \\
& \stackrel{\text{Eq. 10}}{\leq} \sum_{g \in \mathcal{M} \setminus \cup_{j \in (K_1 \cup K_2 \cup K_3)} B_j} \max\{p(g), \min\{v_i(g), TPS_i\}\} \\
& + \sum_{g \in \cup_{j \in K_1} B_j} \left(p(g) - \frac{n}{2n-1} TPS_i \right) \\
& \leq \sum_{g \in \mathcal{M} \setminus \cup_{j \in (K_1 \cup K_2 \cup K_3)} B_j} \bar{v}_i(g) + \sum_{g \in \cup_{j \in K_1} B_j} \left(p(g) - \frac{n}{2n-1} TPS_i \right)
\end{aligned}$$

whereas the RHS of this inequality is a lower bound on the remaining value after k agents were allocated a bundle.

Now, to see that the algorithm terminates, consider $k \leq n-1$. Then, the remaining value is at least:

$$\left(n - \frac{2n \cdot k}{2n-1} \right) \cdot TPS_i \geq \frac{n \cdot (2n-1) - 2n \cdot (n-1)}{2n-1} \cdot TPS_i = \frac{n}{2n-1} TPS_i,$$

and so we are able to fill another bundle. We conclude that we fill n bundles, at which point no active agent remains and the algorithm terminates.

D.2 TPS Approximation With SEFX Allocations

A shrinking operation finds a minimal subset $B' \subseteq B$ of the original bundle B for some agent i , that is of greater value than the bundle they currently hold, and so that no other agent ϵ -SEFX-envies B' . We assign B' to agent i . W.l.o.g., B' requires selling at most one good. If such a good is sold, refer to it as $e_{B'}$. Sale proceeds of $e_{B'}$ that are not assigned to i are kept unassigned. If in some future iteration B' is released (because agent i is envious of another bundle, which is then assigned to agent i), these remaining sale proceeds and the sale proceeds that were included in B' are deleted, and the good $e_{B'}$ is returned to the pool of unallocated goods (undoing the sale of $e_{B'}$). This allows us to ensure that each agent is responsible for at most a single forced sale, from the bundle it is currently allocated (as in Lemma 5).

This, overall, allows us to maintain two important properties of Algorithm 4 for the TPS approximation: (i) Every assigned agent has a bundle of value at least $\frac{n}{2n-1}$ -TPS. This is because the shrunked bundle has at least the value of the existing allocation for a stealing agent, which is of value at least $\frac{n}{2n-1}$ -TPS itself. (ii) Every unassigned agent either has a value of at most $\frac{2n}{2n-1}$ -TPS for an assigned bundle, or that bundle contains at most one good. With these properties maintained, our TPS approximation argument follows through, and we can focus on proving that the allocation is ϵ -SEFX (by Lemma 11, this allows us to find a SEFX allocation as well), and that the algorithm always terminates. One important element of our shrinking operation is that it forces progress by adding something small to the minimal bundle (either ϵ in sale proceeds, or rounding up sale proceeds to a full good). We can motivate the forced progress by the following example:

Example 2. Consider $n = 2$ agents and two goods g_1, g_2 , with valuations and market price as given in Table 3. The good g_2 has high market value (more than $\frac{n}{2n-1} = \frac{2}{3}$ -TPS for both agents), and so following a moving-knife procedure like in Algorithm 4, we allocate $\frac{2}{3}$ in sale proceeds to agent 2 (that has the lower TPS requirement). Still, the remaining $\frac{4}{3}$ in sale proceeds are more than $\frac{2}{3}$ of the TPS of agent 1, and so our next step is to consider 1 in sale proceeds to be allocated to agent 2. If we do the shrinking operation, *without forcing ϵ progress*, agent 1 SEFX-envies any amount of sale proceeds greater than $\frac{2}{3}$, and so they would be allocated $\frac{2}{3}$ of the remaining sale proceeds, and release their current bundle, which is also $\frac{2}{3}$ in sale proceeds. This may then continue indefinitely. If, on the other hand, we force ϵ progress, this process ends up with a partial allocation of 1 in sale proceeds to both agents, which is both $\frac{2}{3}$ -TPS, and SEFX.

	g_1	g_2	TPS
Agent 1	1	0	1.5
Agent 2	0	0	1
Market price p	0	2	–

Table 3. Subjective valuations and market prices for Example 2

A few notes on terminology:

For two bundles $X = (A, P), Y = (A', P')$, we say $X \sqsubseteq Y$ is a *sub-bundle* of Y if $A \subseteq A'$, and for some $g \in A' \setminus A$, $P \leq P' + p(g)$. Notice that because our notion of a generalized bundle contains both full goods and sale proceeds, our notion essentially considers removing proceeds, then selling a good, removing its proceeds, and so on.

In Algorithm 6 we use the term $Unshrink(B')$ to mean the following: Every full good in B' gets added to AVAIL. For a partial good, if there is a “virtual” good in AVAIL that consists of leftovers from the same good, it is joined with it to form a new “virtual good” (or, if it now contains the full good, then the original full good), and if there is no such virtual good, it is added as a virtual good to AVAIL. Notice that due to the operation of Algorithm 6, that leaves all leftovers of a good that was shrunked during bag-filling unassigned, the effect is the following: Any good that was shrunked during bag-filling reappears as the full good after Unshrink. For a good that was shrunked during the large good selling/assignment loop, it depends on whether after Unshrink there are still agents who are allocated parts of that good: If there are, then all leftovers are joined into one virtual good, and if there are no such agents, then the full good is in AVAIL. This maintains the property that a good is divided only if there is an agent that holds part of its proceeds. Notice also that in Shrink (Algorithm 5), we shrink so that only at most a single good is divided. All in all, this guarantees that there are no more goods sold at any step than agents that are currently allocated a bundle, which is a property we need for the TPS approximation.

ALGORITHM 5: Shrink

Input: Bundle X , minimal progress ϵ , valuations $v_1, \dots, v_n$, market price p , Current bundle assignments $B_1, \dots, B_n$

Output: A shrunked bundle X_{shrink} , corresponding agent i'

- 1 Let *ACTIVE* be the agents i so that $B_i = \emptyset$, and *INACTIVE* the remaining agents.
 - 2 Let $X_{shrink} \sqsubseteq X$ be a sub-bundle of X so that there is $i \in ACTIVE$ so that $\bar{v}_i(X_{shrink}) \geq \frac{n}{2n-1}TPS_i$. Let i' be the corresponding agent.
 - 3 **if** $\exists i \in INACTIVE$ so that i ϵ -SEFX-envies X_{shrink} **then**
 - 4 Let $X_{shrink} \sqsubseteq X$ be a maximal bundle so that no agent SEFX-envies any strict sub-bundle of it. // We can always find X_{shrink} by repetitively following the SEFX-envy violation and either selling a good or removing proceeds accordingly.
 - 5 Let $\delta = \max_{i \in INACTIVE} \bar{v}_i(X_{shrink}) - \bar{v}_i(B_i)$, and let agent i be a maximizing agent.
 - 6 **if** $\delta < \epsilon$ **then**
 - 7 If a good g has leftover sale proceeds of at least $\epsilon - \delta$, add the sale proceeds to X_{shrink} . If there is no such good, round up (add all the remaining sale proceeds) from some sold good g .
 - 8 **end**
 - 9 **end**
 - 10 Return X_{shrink}, i' .
-

ALGORITHM 6: APX-TPS- ϵ -SEFX**Input:** Valuations $v_1, \dots, v_n$, market price function p over goods $\mathcal{M}$, number of agents n **Output:** A partial allocation $\mathbf{B}$ with at least $\frac{n}{2n-1} \cdot TPS_i$ utility for each agent, no ϵ -SEFX envy

```

1   $AVAIL = \mathcal{M}, ACTIVE = [n], \forall i, A_i = \emptyset.$ 
2  while  $|ACTIVE| \neq 0$  do
3      while  $\exists j \in AVAIL, i \in ACTIVE, p(j) > \frac{n}{2n-1} TPS_i$  do
4          Let  $i^* \in \arg \min_{i \in ACTIVE} TPS_i.$ 
5          Let  $\alpha$  be  $\frac{n}{2n-1} \cdot TPS_{i^*}$  in sale proceeds from  $j.$ 
6           $X_{shrink}, i' = \text{Shrink}(\{j\}, \epsilon, \mathbf{v}, p, \mathbf{B})$ 
7           $AVAIL = AVAIL \setminus \{X_{shrink}\}$ 
8          if  $i' \notin ACTIVE$  then
9               $AVAIL = AVAIL \cup \{B_{i'}\}$  // add  $i'$  current bundle back to the pool
10         end
11         else
12              $ACTIVE = ACTIVE \setminus \{i'\}$ 
13         end
14          $B_{i'} = X_{shrink};$  //  $i'$  gets the shrunked bundle
15     end
16     while  $\exists j \in AVAIL, i \in ACTIVE, v_i(j) > \frac{n}{2n-1} TPS_i$  do
17          $X_{shrink}, i' = \text{Shrink}(\{j\}, \epsilon, \mathbf{v}, p, \mathbf{B})$ 
18          $AVAIL = AVAIL \setminus \{X_{shrink}\}$ 
19         if  $i' \notin ACTIVE$  then
20              $AVAIL = AVAIL \cup \{B_{i'}\}$  // add  $i'$  current bundle back to the pool
21         end
22         else
23              $ACTIVE = ACTIVE \setminus \{i'\}$ 
24         end
25          $B_{i'} = X_{shrink};$  //  $i'$  gets the shrunked bundle
26     end
27     Bag-fill complete goods into a bundle  $X$ , until some agent  $i \in ACTIVE$  values it more than  $\frac{n}{2n-1} TPS_i.$ 
28      $X_{shrink}, i' = \text{Shrink}(X, \epsilon, \mathbf{v}, p, \mathbf{B})$ 
29      $AVAIL = AVAIL \setminus X.$ 
30     if  $i' \notin ACTIVE$  then
31          $AVAIL = AVAIL \cup \{Unshrink(B_{i'})\}$ 
32     end
33     else
34          $ACTIVE = ACTIVE \setminus \{i'\}$ 
35     end
36      $B_{i'} = X_{shrink};$ 
37 end

```

Theorem 2. Every allocation instance has a $\frac{n}{2n-1}$ -TPS allocation. Moreover, every allocation instance has an allocation that is both $\frac{n}{2n-1}$ -TPS and SEFX.

Proof. Any agent i that is assigned a bundle during the run of the algorithm gets at least $\frac{n}{2n-1} TPS_i$. This is since it can be assigned a bundle for the first time either in the large goods assignment loop, or in the bag-filling loop, whereas in each of these cases it has a value of at least $\frac{n}{2n-1} MMS_i$ for the bundle. Then in any subsequent reallocation for the agent, its value increases.

We show that every agent gets allocated a bundle. First, we note we cannot “get stuck” in the large good selling/assignment loop. The reason is that at each iteration, either another agent gets allocated a bundle, or that *Shrink* allocates to an agent i' that is already allocated a bundle, so that it receives an additional $\epsilon \min_i TPS_i$ in value, or a full good. We thus always make a Pareto improvement w.r.t. the agents that are already allocated a bundle, with either at least ϵ of additional value, or along a finite set (of the possible item-agent matchings). Similarly, we cannot be stuck in the general allocation loop. In the general allocation loop, the bag-filling step is always completed

successfully. This is because the Shrink operation only makes bags less valuable in the perspective of agents that are not assigned them, compared with if the bag was not shrunk. Thus, at any iteration of the bag-filling loop, the agents that are not assigned a bundle satisfy the conditions of Theorem 2, and so we are guaranteed that we can continue filling bags. Given that, since we make a Pareto improvement w.r.t. the agents that are already allocated a bundle, with either at least ϵ of additional value, or along a finite set (of the possible item-agent matchings), the loop terminates with all agents assigned a bundle.

The ϵ -SEFX property is maintained inductively among the agents that are allocated a bundle at any step of the algorithm. Initially, no agent is allocated a bundle, so it holds in a vacuous sense. At each step when an agent is allocated a bundle (or reallocated a bundle), the shrinking operation ensures the property is maintained.

Then, Lemma 11 shows that since we can guarantee a partial allocation with $\frac{n}{2n-1}$ -TPS together with ϵ -SEFX for any ϵ , we can also guarantee a partial allocation with $\frac{n}{2n-1}$ -TPS together with SEFX. Then, we can use the process of [9] (that we also use in Lemma 10) to extend it to a full allocation with the same properties.

E Pseudo-Polynomial Algorithm and PTAS for MMS Computation

It is well-known that computing the MMS in the setting of additive indivisible goods, with a fixed number of agents, even with $n = 2$, is weakly NP-hard, by a reduction from PARTITION [14]. Weakly NP-hardness holds in our setting as well. Complementing this hardness result, we show how to compute the MMS in pseudo-polynomial time (a result previously known for the setting without sellable goods).

E.1 Pseudo-Polynomial Algorithm for $n = 2$

We start by considering the $n = 2$ (two agents) case. Formally, we assume that v, p take integer values in $\{0, \dots, K\}$ for some K , and we require that the algorithm is polynomial in the number of goods m and K . Our main building block is the well-known dynamic programming algorithm for MMS computation with indivisible goods (and no selling), which we include here for completeness:

ALGORITHM 7: INDIVISIBLE-MMS-COMP

Input: Valuation v in range $[0, K]$ over goods $\mathcal{M}$

Output: The MMS with Indivisible Goods

```

1 Let  $\sigma = \sum_{g \in \mathcal{M}} v(g)$ 
2 Let  $S = \emptyset$ ;
3 Let  $T : \{0, \dots, \lfloor \frac{\sigma}{2} \rfloor\} \times \{0, \dots, m+1\} \rightarrow \{0, 1\}$ , and initialize  $\forall j, T(0, j) = 1$ .
4 for  $j = 1$  to  $m$  do
5   for  $k = 1$  to  $\lfloor \frac{\sigma}{2} \rfloor$  do
6      $T(k, j) = T(k, j-1) \vee T(k - v(g_j), j-1)$ ;
7   end
8 end
9 return  $\arg \max_c T(c, m) = 1$ .
```

The algorithm works as follows. It looks for the highest number smaller than $\frac{\sigma}{2}$, where σ is the total sum of values, that can be attained by a subset of $\mathcal{M}$. This number T will be the MMS, as its complementary will be $\sigma - T \geq \frac{\sigma}{2} \geq T$. The search is done by adding a new good for consideration at each iteration, and expanding the set of valid sums by considering each good in the set (which

corresponds to a sum achieved by a subset) and either adding the good (and $v(g_j)$ to the sum) or not adding the good (and maintaining the existing sum of the subset as an option). The following lemma, which we state without proof, summarizes the guarantee of INDIVISIBLE-MMS-COMP.

Lemma 13. *INDIVISIBLE-MMS-COMP runs in time $O(m^2 \cdot K)$ and returns the MMS for indivisible goods without selling.*

ALGORITHM 8: COMP-MMS- $n=2$

Input: Valuation v and market price p in range $[0, K]$ over goods $\mathcal{M}$

Output: MMS value for v, p with $n = 2$

```

1 Let  $\forall g, \bar{v}(g) = \max\{v(g), p(g)\}$ .
2  $currentMMS = \text{INDIVISIBLE-MMS-COMP}(\bar{v})$ ;
3 for  $j = 1$  to  $m$  do
4    $v^{-j}(g) = \begin{cases} \bar{v}(g) & g \neq j; \\ 0 & g = j; \end{cases}$ 
5    $base = \text{INDIVISIBLE-MMS-COMP}(v^{-j})$ ;
6   if  $base \geq \frac{1}{2} \left( \sum_{g \in \mathcal{M} \setminus \{j\}} \bar{v}(g) - p(j) \right)$  then
7      $currentMMS = \max\{currentMMS, \frac{1}{2} \left( \sum_{g \in \mathcal{M} \setminus \{j\}} \bar{v}(g) + p(j) \right)\}$ 
8   end
9 end
10 return  $currentMMS$ ;

```

Lemma 14. *COMP-MMS- $n=2$ runs in time $O(m^3 \cdot K)$ and returns the MMS value given v, p .*

Proof. The running time holds since we perform a loop with m steps where the main component is INDIVISIBLE-MMS-COMP, which runs in $O(m^2 \cdot K)$ by Lemma 13.

Regarding correctness, recall that by Lemma 5, with $n = 2$ goods, it is w.l.o.g. to assume that there are at most one split good. We can therefore compute the MMS value conditional on each certain good being a split good, or no split goods at all, and take the maximum over these options. Moreover, by Lemma 7, the MMS value with split good g_j equals exactly $MMS^j = \frac{1}{2} \left(\sum_{g \in \mathcal{M} \setminus \{j\}} \bar{v}(g) + p(j) \right)$. However, we must verify that this is indeed attainable. For this value to be attainable, we must be able to partition the goods other than j into two bundles that are at most $p(j)$ apart. Then, we can use $p(j)$ to ensure that each bundle has the same value once we take the split good into consideration. Thus, the MMS value for the split good g_j can be attained if and only if INDIVISIBLE-MMS-COMP over all goods other than g_j is at least $MMS^j - p(j)$. This is the exact logic followed by COMP-MMS- $n=2$.

E.2 Pseudo-Polynomial Algorithm for General Fixed n

We can further develop the argument above to compute the MMS in pseudo-polynomial time for any fixed n . Notice that since 3-PARTITION is strongly NP-hard, we cannot expect a pseudo-polynomial algorithm when n is part of the input, unless $P = NP$.

For this purpose, we must first significantly generalize INDIVISIBLE-MMS-COMP in the following way: While with $n = 2$, the sum of goods smaller than $\frac{K}{2}$ uniquely determines its complement, with general n , our dynamic programming table is $n + 1$ -dimensional (one dimension for the number of goods used, corresponding to the columns in the $n = 2$ case, and n dimensions for the

sum-structure of a partition to n bundles). Moreover, in this case, we incorporate the divisible sale proceeds p into Algorithm 10 as an additional input. After computing the dynamic programming table, for every achievable sum-structure, we translate it into a single number by a waterfall procedure using p . In the waterfall procedure, we are given bundle sum values, and “water” that can be distributed among them, and we gradually fill the bundles from the lowest to highest value to achieve the highest possible MMS value given the input.

ALGORITHM 9: WATERFALL-FILL

Input: $S_1, \dots, S_n$ integer sums, integer p in range $[0, m \cdot K]$

Output: The maximin value of dividing sale proceeds p to complement sums $S_1, \dots, S_n$

```

1 Sort  $S_1, \dots, S_n$  in ascending order;
2  $r = p$  //
3 for  $j = 1$  to  $n - 1$  do
4   if  $r \geq j \cdot (S_{j+1} - S_j)$  then
5     for  $j' = 1$  to  $j$  do
6        $S_{j'} = S_{j+1}$ 
7     end
8      $r = r - j \cdot (S_{j+1} - S_j)$ 
9   end
10  else
11    return  $S_1 + \frac{r}{j}$ 
12  end
13 end
14 return  $S_1 + \frac{r}{n}$ 

```

Recall that the $\tilde{O}$ notation means we disregard terms with logarithmic dependence on the problem parameters. The following lemma shows that waterfall-fill optimally distributes the “water” p .

Lemma 15. *WATERFALL-FILL runs in $\tilde{O}(n)$ (as a problem with parameters n, m, k) and has the highest MMS value for an instance with bundle sums $S_1, \dots, S_n$ and “water” p .*

Proof. The running time is due to the main loop which runs at most $n - 1$ times and performs simple arithmetic operations (which account for a logarithmic dependence on m, K , subsumed by the $\tilde{O}$ notation).

Correctness holds by induction over the step where the loop terminates. The inductive claim is that if we denote $S_1^{(j)}, \dots, S_n^{(j)}$ for the sums after step j (if no result is returned), then $S_1^{(j)} = \dots = S_j^{(j)}$, as well as $\forall j' > j, S_{j'}^{(j)} = S_{j'}$, and that the best attainable MMS given $S_1^{(j-1)}, \dots, S_n^{(j-1)}$ and sale proceeds $p - r$ is $S_1^{(j)}$ (if the loop continues) or the return value, otherwise.

For the base case, if the loop terminates after one step, then $p < S_2 - S_1$, and thus

$$S_1 + p < S_2 \leq S_3 \leq \dots \leq S_n,$$

and so if we add p sale proceeds to bundle 1 we attain an MMS of $S_1 + p$. On the other hand, for any $p_1, \dots, p_n \geq 0$ so that $\sum_{i=1}^n p_i = p$, it holds that

$$\min\{S_1 + p_1, \dots, S_n + p_n\} \leq S_1 + p_1 \leq S_1 + p,$$

and so this is the best attainable MMS given $S_1, \dots, S_n, p$. If the loop continues after the first step, then $p - r = S_2 - S_1, S_1^{(1)} = S_2$, and we similarly conclude that this is the best attainable MMS.

For the inductive step, assume that $S_1^{(j)} = \dots = S_j^{(j)}$. If the loop terminates at step $j + 1$, then $r < j \cdot (S_{j+1} - S_j)$, and thus

$$S_1^{(j)} + \frac{r}{j} = S_2^{(j)} + \frac{r}{j} = \dots = S_j^{(j)} + \frac{r}{j} < S_{j+1}^{(j)} \leq \dots \leq S_n^{(j)},$$

and so if we add $\frac{r}{j}$ sale proceeds to each of the first j bundles we attain an MMS of $S_1^{(j)} + \frac{r}{j}$. On the other hand, for any $p_1, \dots, p_n \geq 0$ so that $\sum_{i=1}^n p_i = r$, it holds that

$$\min\{S_1 + p_1, \dots, S_n + p_n\} \leq \min_{1 \leq j' \leq j} S_{j'}^{(j)} + p_{j'} = S_1^{(j)} + \min_{1 \leq j' \leq j} p_{j'} \leq S_1^{(j)} + \frac{r}{j},$$

and so this is the best attainable MMS given $S_1^{(j)}, \dots, S_n^{(j)}, r$. If the loop continues after step j , then $r^{(j)} - r^{(j+1)} = j \cdot (S_{j+1}^{(j)} - S_j^{(j)})$, $S_1^{(j+1)} = \dots = S_{j+1}^{(j+1)}$, and we similarly conclude that this is the best attainable MMS given $S_1^{(j)}, \dots, S_n^{(j)}, r^{(j)} - r^{(j+1)}$.

WATERFALL-FILL receives both the divisible resource p and specific bundle sums. We next abstract one step further, and not include the bundle sums as input, but rather good values, together with a divisible resource p . Then, as we noted above, we combine a dynamic programming approach (as done in INDIVISIBLE-MMS-COMP for $n = 2$) for general n , with an application of WATERFALL-FILL to handle the divisible resource. This results in Algorithm 10.

ALGORITHM 10: MAX-SUM-ACHIEVABLE-GENERAL- n

Input: Number of agents n , valuation v in range $[0, K]$ over goods $\mathcal{M}$, integer sale proceeds p

Output: The MMS value given n, v, p

```

1 Let  $S = \emptyset$ ;
2 Let  $T : \{0, \dots, m \cdot K\}^n \times \{0, \dots, m + 1\} \rightarrow \{0, 1\}$ , and initialize  $\forall j, T(0, \dots, 0) = 1$ .
3 for  $j = 1$  to  $m$  do
4   for  $s_1, \dots, s_n \in \{0, \dots, m \cdot K\}^n$  do
5      $T(s_1, \dots, s_n, j) =$ 
6        $T(s_1 - v(g_j), s_2, \dots, s_n, j - 1) \vee T(s_1, s_2 - v(g_j), \dots, s_n, j - 1) \vee \dots \vee T(s_1, s_2, \dots, s_n - v(g_j), j - 1)$ 
7   end
8  $currentMMS = 0$ 
9 for  $s_1, \dots, s_n \in \{0, \dots, m \cdot K\}^n$  do
10   if  $T(s_1, \dots, s_n, m) = 1$  then
11      $currentMMS = \max\{currentMMS, WATERFALL - FILL(s_1, \dots, s_n, p)\}$ 
12   end
13 end
14 return  $currentMMS$ 
```

Lemma 16. *For a given valuation function v over goods $\mathcal{M}$, and divisible sale proceeds p , MAX-SUM-ACHIEVABLE-GENERAL- n returns the conditional MMS value in time $\tilde{O}((m \cdot K)^n \cdot n \cdot m)$.*

Proof (Proof sketch.). The running time is due to the algorithm running over an array of size $(m \cdot K)^n \cdot m$ and performing a $\tilde{O}(n)$ operation at each step.

The correctness holds since the algorithm takes the maximum over the WATERFALL-FILL value, which guarantees the best attainable MMS given sums $S_1, \dots, S_n$ and divisible sale proceeds p , over all feasible sums, given by the dynamic program.

MAX-SUM-ACHIEVABLE-GENERAL- n finds the MMS given a particular selling decision, which translates an instance to goods with value v , and a divisible resource p . Finally, we make the final abstraction to consider the original instance with a valuation v and market price p over m goods in COMP-MMS-FIXED- n . We make use of Lemma 5 in that we only need to consider subsets of size up to $n - 1$ of goods that are sold in the MMS partition, and use $\bar{v}$ for the remaining goods.

ALGORITHM 11: COMP-MMS-FIXED- n

Input: Number of agents n , valuation v and market price p in range $[0, K]$ over goods $\mathcal{M}$

Output: MMS value for n, v, p

```

1 Let  $\forall g, \bar{v}(g) = \max\{v(g), p(g)\}$ .
2  $currentMMS = 0$ 
3 for  $s = 0$  to  $n - 1$  do
4   for  $S = \{j_1, \dots, j_s\} \subseteq \mathcal{M}, s.t. |S| = s$  do
5      $currentMMS = \max\{currentMMS,$ 
6        $MAX-SUM-ACHIEVABLE-GENERAL-n(n, \bar{v}, \mathcal{M} \setminus S, \sum_{g \in S} p(g))\}$ 
7   end
8 end
9 return  $currentMMS$ ;

```

Lemma 17. *COMP-MMS-FIXED- n computes the MMS for an instance of goods with selling in pseudo-polynomial time $\tilde{O}((\frac{e \cdot m}{n} \cdot m \cdot K)^{n-1} \cdot n \cdot m)$.*

Proof. The running time follows by iterating over all choices of up to $n - 1$ goods out of m , where we can upper bound the binomial sum [5]:

$$\sum_{s=0}^{n-1} \binom{m}{s} \leq \left(\frac{e \cdot m}{n-1}\right)^{n-1}.$$

This is multiplied by the running time of MAX-SUM-ACHIEVABLE-GENERAL- n in each iteration, which is given by Lemma 16.

Regarding correctness, we use the fact that by Lemma 5, an MMS output for n agents has, w.l.o.g., at most $n - 1$ split goods. Then, for any fixed choice of split goods $\mathcal{S}_C$, we sell them, and MAX-SUM-ACHIEVABLE-GENERAL- n guarantees (through Lemma 16) the best conditional MMS guarantee given the indivisible goods $\mathcal{M} \setminus \mathcal{S}_C$ and the divisible sale proceeds $\sum_{g \in \mathcal{S}_C} p(g)$. Taking the maximum over all these options results in the overall MMS value.

E.3 A Polynomial-Time Approximation Scheme (PTAS) for MMS Computation for Goods With Selling

We show how to reduce an instance of goods with selling to an instance of indivisible goods (without selling) for the purpose of a PTAS. We can then rely on the known PTAS for goods without selling [37].

Theorem 5. *For the purpose of computing the MMS value and an associated MMS partition, given a PTAS for indivisible goods without selling, there is a PTAS for goods with selling.*

Proof (Proof sketch). Let ALG denote a PTAS for goods without selling.

In the instance with selling, we sell all goods with $p(g) \geq v(g)$, and iteratively sell all goods with $p(g)$ higher than the TPS (recall that the TPS is computable in polynomial time). None of

these sales can decrease the MMS. By multiplicative scaling, we may assume that the TPS of the resulting instance is 1. This also implies that the MMS is at least $\frac{1}{2}$. (This relation between MMS and TPS is known when there are no sellable goods. Its proof, by a simple bucket-filling algorithm, applies also when there are sellable goods.)

Choose $k \simeq \frac{1}{\epsilon} \log \frac{1}{\epsilon}$, so that $\epsilon(1 + \epsilon)^k = 1$ (if needed, tweak the value of ϵ so that k with this property exist). For every unsold good, we place it in one of $k + 2$ buckets. Bucket B_0 contains goods with $v(g) \leq \epsilon$. For $1 \leq i < k$, bucket B_i contains goods with $\epsilon \cdot (1 + \epsilon)^{i-1} < v(g) \leq \epsilon \cdot (1 + \epsilon)^i$. Bucket B_k contains those goods with $v(g) > 1$.

For the purpose of obtaining an MMS partition, the benefit of selling goods for B_0 is negligible, as they never cause an imbalance larger than an additive term of ϵ among the bundles of the MMS partition.

By Lemma 5, we need to sell at most $n - 1$ unsold goods. A *configuration* specifies how many goods from each of the $k + 1$ buckets $B_1, \dots, B_{k+1}$ need to be sold. The total number of possible configurations is at most n^{k+1} . Given a configuration $s = s_1, \dots, s_{k+1}$, the s_j goods that we sell from each bucket B_j are those that have the highest price, because in terms of v value, there is negligible difference among goods in the same bucket. This applies also to goods in B_{k+1} , because any unsold good in B_{k+1} has value larger than the MMS, and for the purpose of achieving an MMS partition, they are interchangeable.

For a given configuration, we group the money obtained from selling goods into small auxiliary indivisible goods of value ϵ each, and add them to the instance. There are at most $O(\frac{m}{\epsilon})$ auxiliary goods.

Then, for each configuration we run ALG. Our output approximation for the MMS (and the MMS-partition) is the one given by the configuration for which ALG gave the highest MMS value.

It is not hard to see that the algorithm sketched above gives a $(1 - O(\epsilon))$ approximation to the MMS. In terms of its running time, it calls ALG with each of at most $n^{k+1} \leq n^{f(\epsilon)}$ configurations, each time with a number of goods that is $O(\frac{m}{\epsilon})$. Here, $f(\epsilon) = \frac{1}{\epsilon} \log(\frac{1}{\epsilon})$. As ALG is a PTAS, so is our algorithm.

F Extension: Chores with Outsourcing

It is natural to translate our model to the chores setting, which we call “Chores with Outsourcing”. In the chores setting, agents have subjective costs of performing different tasks, but there is also an option to pay a known cost for the task to be outsourced. The cost can be divisibly shared between the agents. We do not restate the entire model, but rather use c (cost) functions instead of v (valuation) functions notation, and t (task) instead of g (good) for the goods. We let

$$\bar{c}_i(t) = \min\{c_i(t), p(t)\}.$$

Notice that c and p receive positive values, representing the *dis-utility* of the allocation (which we aim to minimize).

How do our results translate to this setting? The $n = 2$ case goes through well, and allows us to guarantee the MMS, using an analogous procedure to that of the goods with selling setting.

F.1 2-MMS Approximation for Chores

We need to consider throughout the algorithm that outsourced chores may be very costly relative to the MMS. This is potentially a problem, if we consider, as a starting point, the following natural algorithm for indivisible chores (without outsourcing). The algorithm is an adaptation of the bag-filling algorithm in [6], which was given for the goods setting, and gives a 2-approximation.

ALGORITHM 12: APX-MMS-Indivisible-Chores-2**Input:** Cost functions $c_1, \dots, c_n$ over chores $\mathcal{M}$, number of agents n **Output:** An allocation B with at most $2 \cdot MMS$ cost for each agent

```

1 for  $j = 1$  to  $n$  do
2   Let  $B_j = \emptyset$ .
3   Do bag-filling for  $B_j$  until all agents agree  $c_i(B_j) \geq MMS_i$ .
4   Allocate  $B_j$  to an agent that has  $c_i(B_j) \leq 2 \cdot MMS_i$ .
5 end

```

The algorithm achieves the 2-approximation because of an important property for indivisible chores MMS: No chore costs more than the MMS cost. If a chore were to cost more, then in any partition, the bundle that contains it would cost more than the MMS, including in the MMS partition, which is a contradiction. Thanks to this property, we are guaranteed that if for some agent a bundle's dis-utility is at most the MMS, then the dis-utility after adding any chore is at most $2 \cdot MMS$. This allows us to always be able to fill a bag that is worth at least the MMS cost for all agents, but still have an agent that values it at most at $2 \cdot MMS$. However, as noted before, this property is no longer true in the chores with outsourcing setting, if we consider allocating complete outsourced tasks (i.e., adding the full costs of an outsourced tasks within a bundle).

Still, with a careful adjustment and analysis, it is possible to achieve the 2-approximation. The idea is as follows. At each round of bag-filling, we put aside chores that cause issues in the bag-filling process, i.e., before adding them some agents consider the bundle has below MMS_i cost, and after adding them all agents think the bundle has above $2MMS_i$ cost. If we end up putting all chores aside (except for those already in the bag) and are unable to complete the bag-filling as we wish, then we allocate the bundle as it is, and continue to the next round. This violates the bag-filling principle, in that some agents may consider the cost of the allocated bundle is too low (below MMS_i). However, the gist of the proof is that this violation only happens at most once in the perspective of any agent i , and this still allows for the argument to follow through.

Lemma 18. *APX-MMS-Chores-With-Outsourcing-2 guarantees a 2-approximation for all agents.*

Proof. All chores are allocated by the end of the algorithm, because either a bundle is allocated at each round of the for loop, or there is a round where all remaining chores are outsourced and the sale proceeds are split among the active agents. If the latter happens, then clearly all chores are allocated. If the former happens, then the remaining agent in ACTIVE takes all remaining chores.

Let us consider the approximation guarantee. It is clear by the IF conditions that any agent allocated a bundle within the bag-filling loop (rather than shares the outsourcing cost) has a cost of at most $2 \cdot MMS$. We thus focus on the last agent that is allocated a bundle outside the loop, or on the agents that share the outsourcing cost (the two possible return conditions, of line 17 and 20).

Let us first consider the last agent i that is allocated a bundle in line 20. If all bundles of other agents have a cost of at least $\bar{c}_i(X) \geq MMS_i$, then overall, $\bar{c}_i(\cup_{j \neq i} B_j) = \sum_{j \neq i} \bar{c}_i(B_j) \geq (n-1) \cdot MMS_i$. Since it also holds that $n \cdot MMS_i \geq \bar{c}_i(\mathcal{M})$, we conclude that $\bar{c}_i(\mathcal{M} \setminus (\cup_{j \neq i} B_j)) \leq MMS_i$.

If for some other agent j it holds that $\bar{c}_i(B_j) < MMS_i$, then it must be allocated a bundle within the second IF condition in line 14 (as it is prohibited by the pre-condition of the first IF condition). Take the first such bundle that is allocated within the second IF condition. This means that when agent j is allocated bundle B_j , it holds that $B_j \cup \Psi = AVAIL$, and for any $t \in \Psi$, $\bar{c}_i(B_j \cup \{t\}) > 2MMS_i$. We thus conclude that $\bar{c}_i(t) > MMS_i$ for any $g \in \Psi$. Since Ψ is all the available chores from this round on-wards, $\bar{c}_i(t) > MMS_i$ subsequently holds in all

ALGORITHM 13: APX-MMS-Chores-With-Outsourcing-2**Input:** Costs $c_1, \dots, c_n$ and outsourcing price p over chores $\mathcal{M}$, number of agents n **Output:** An allocation $\mathbf{B}$ with at most $2 \cdot MMS$ cost for each agent

```

1 Let  $AVAIL = \mathcal{M}$  be all the available goods,  $ACTIVE = [n]$  all the active agents.
2 for  $j = 1$  to  $n - 1$  do
3   Let  $X = \emptyset, \Psi = \emptyset$ . //  $X$  is the bundle we bag-fill.  $\Psi$  are all the chores we take out of circulation for the
   current round of bag-filing.
4   while  $\Psi \neq AVAIL \setminus X$  do
5     Add chores to  $X$  from  $AVAIL \setminus (X \cup \Psi)$ , until all active agents  $i$  agree that  $\bar{c}_i(X) \geq MMS_i$ . Let  $t$  be
     the last chore added to  $X$ .
6     if There is an agent  $i$  so that  $\bar{c}_i(X) \leq 2 \cdot MMS_i$  then
7       |  $B_i = X, ACTIVE = ACTIVE \setminus \{i\}, AVAIL = AVAIL \setminus B_j$ .
8     end
9     else
10      |  $\Psi = \Psi \cup \{t\}, X = X \setminus \{t\}$ .
11    end
12  end
13  if  $X \neq \emptyset$  then
14    | Allocate  $X$  to an agent  $i$  that has  $c_i(X) \leq 2 \cdot MMS_i$ , and let
    |  $ACTIVE = ACTIVE \setminus \{i\}, AVAIL = AVAIL \setminus X$ .
15  end
16  else
17    | Outsource all chores in  $AVAIL$ , share equally among all  $ACTIVE$  agents, and return.
18  end
19 end
20 Allocate all chores in  $AVAIL$  to the last remaining agent in  $ACTIVE$ , and return.

```

future rounds for any chore in $AVAIL$. This means that any other bundle X that is allocated, since it contains some chore $t \in AVAIL$, has $\bar{c}_i(X) > MMS_i$. Overall, we conclude that for any other agent $\ell \neq j$, $\bar{c}_i(B_\ell) \geq MMS_i$, and we can repeat our previous argument to show that $\bar{c}_i(\mathcal{M} \setminus (\cup_{\ell \neq i} B_\ell)) \leq \bar{c}_i(\mathcal{M} \setminus (\cup_{\ell \neq \{j, i\}} B_\ell)) \leq n \cdot MMS_i - (n - 2)MMS_i = 2MMS_i$.

Now consider if the outsourcing step of line 17 takes place. This step only happens if $X = \emptyset$. Since the condition happens after the While loop is terminated, it must be that $\Psi = AVAIL \setminus X = AVAIL$. Thus, all remaining agents consider all remaining chores $t \in AVAIL$ to have $\bar{c}_i(t) > 2MMS_i$ (otherwise, either the IF condition of line 6 would be invoked, or X would not be empty). Since a chore that has $\bar{c}_i(t) > MMS_i$ must be outsourced in the MMS partition, we conclude that all agents outsource the remaining chores in their MMS partition. Following our previous arguments, for any agent $i \in ACTIVE$, it holds that if k rounds of bag-filling have been previously completed, $p(AVAIL) + \bar{c}_i(\cup_{j=1}^k B_j) = p(AVAIL) + \sum_{j=1}^k \bar{c}_i(B_j) \leq n \cdot MMS_i$, and $\sum_{j=1}^k \bar{c}_i(B_j) \geq (k-1)MMS_i$, and so $p(AVAIL) \leq (n-k+1)MMS_i$. k rounds previously completed also implies there are also $n-k$ agents in $ACTIVE$. Overall, each agent is assigned outsourcing costs of $\frac{n-k+1}{n-k} MMS_i \leq \frac{3}{2} MMS_i$, where the last inequality holds because $1 \leq k \leq n-2$.

F.2 Envy Notion Definitions For Chores

We provide a definition of envy notions for chores.

Definition 9. A chores allocation $(A_1, P_1), \dots, (A_n, P_n)$ is envy free (EF) if for every agent i and other agent j , $\bar{c}_i(A_i) + P_i \leq \bar{c}_i(A_j) + P_j$. An allocation satisfies the following relaxations of EF if the above envy free condition holds whenever $P_i > 0$, and either the EF condition or the following conditions hold if $P_i = 0$.

1. *SEF1 (strong envy free up to one chore)*. $p(t) - c_i(t) \geq \bar{c}_i(A_i) - \bar{c}_i(A_j) - P_j$ for some chore $t \in A_i$.
2. *SEFX (strong/sellable envy free up to any chore)*. $p(t) - c_i(t) \geq \bar{c}_i(A_i) - \bar{c}_i(A_j) - P_j$ for every chore $t \in A_i$.

At a surface level, this definition looks very unlike the EF1 definition for chores (where you remove one chore from the agent's bundle, and compare with the other agent's bundle). We explain why it makes intuitive sense. The immediate analogue in the settings of chores with outsourcing, would be to remove a chore, and replace it with its outsourcing costs. However, this would be the same, or worse, than $\bar{c}_i$ of that chore, and so this would yield a notion equivalent to EF.

A better way to go about it is to consider the local pairwise maximin principle that guided us in the goods with selling case. There, an agent was willing to accept having a worse-off bundle, if they know that a localized change (selling the good) makes the other bundle worse-off. Otherwise, it could be argued that a localized change can improve the maximin of the two bundles: We sell a good of the other bundle, recover the same value as B_i , and are left with some more proceeds that can be shared equally among B_i and B_j .

Similarly, in the chores with outsourcing setting, for SEFX to hold, we should forbid the following scenario: We outsource a chore t from B_i , use water-filling to levy the costs on B_j and $B_i \setminus \{t\}$, and end-up with a lower cost than B_i , since this would constitute a localized maximin improvement. As an inequality, this takes the form:

$$\frac{\bar{c}_i(B_i \setminus \{t\}) + \bar{c}_i(B_j) + p(t)}{2} \geq \bar{c}_i(B_i).$$

By changing wings, we can translate it to the condition:

$$p(t) - \bar{c}_i(t) \geq \bar{c}_i(B_i) - \bar{c}_i(B_j).$$

Now, if $\bar{c}_i(t) = p(t)$, this is just the envy-freeness condition. Otherwise, it must be that $\bar{c}_i(t) = c_i(t)$, and so:

$$p(t) - c_i(t) \geq \bar{c}_i(B_i) - \bar{c}_i(B_j).$$

This yields the nice “delta” form of Definition 9: The difference between outsourcing and in-house costs should be greater than the envy towards the other agent's bundle.

G Extension: Equal Proceeds Case

Our goal in this section is to consider the impact of imposing an additional restriction over the allocation, where we require that sale proceeds are equally distributed among all agents. This may be due to a regulatory or legal constraint, that attempts to impose fairness on the allocation. As the sale proceeds have global objective value, it stands to reason that they should be equally distributed, but as we see, this may significantly worsen the overall fairness of the allocation.

Given the additional restriction, we adjust the MMS value an agent may expect, as it can only come from allocations with equal sale proceeds. This may significantly reduce the MMS the agent expects.

Example 3. Consider $m = n$ goods, in the perspective of some fixed agent with the valuations and market price given in Table 4. Then, the equal-proceeds MMS value is $\frac{1}{n}$, while the unconstrained MMS value is 1.

	g_1	$\dots$	g_{n-1}	g_n
Subjective Valuation v	1	$\dots$	1	0
Market price p	0	$\dots$	0	1

Table 4. Subjective valuation and market price for Example 3

The following example shows that no more than a $\frac{1}{n}$ approximation of the equal-proceeds MMS can be guaranteed.

Example 4. Consider $m = n$ goods, with the valuations and market price given in Table 5. For agent 1, the optimal outcome is selling all goods, resulting in $MMS_1 = 1$. For all other agents, the optimal outcome is keeping all goods, each in a singleton bundle, resulting in $MMS_i = L$.

Now, consider the possible outcomes. If at most one good is sold, then agent 1 gets at most $1 \cdot \frac{1}{n} + (n-1) \cdot \frac{1}{L} \approx \frac{1}{n} \cdot MMS_1$, when L is sufficiently large. If at least two goods are sold, then some agent $i > 1$ does not get to keep any good, by the pigeonhole principle. They get $\frac{2}{n} = \frac{2}{n \cdot L} MMS_i$, which is arbitrarily low with large L .

	g_1	$\dots$	g_n	MMS
Agent 1	$\frac{1}{L}$	$\dots$	$\frac{1}{L}$	1
Agent 2	L	$\dots$	L	L
$\dots$				
Agent n	L	$\dots$	L	L
Market price p	1	$\dots$	1	–

Table 5. Subjective valuations and market prices for Example 4. We consider $L \geq 1$.

We show that this is tight using a bag-filling algorithm that guarantees $\frac{1}{n}$ of the TPS, as defined in Definition 3. Notice that this is a definition of the TPS with unconstrained proceeds, which is always larger than the MMS with unconstrained proceeds, by Lemma 1. Thus, this shows that there is no “double discounting”: Example 3 shows the equal-proceeds MMS can be $\frac{1}{n}$ of the unconstrained MMS, and Example 4 shows we can only guarantee $\frac{1}{n}$ of the equal-proceeds MMS. However, these are two different examples, and so our result shows they cannot be combined to create a $\frac{1}{n^2}$ gap.

Let us describe the bag-filling algorithm. We use the notation

$$v'_i(e) = \max[\min[TPS_i, v_i(e)], p(e)].$$

Let $S_i = \{e \mid p(e) > \min[TPS_i, v_i(e)]\}$, $K_i = \mathcal{M} \setminus S_i$, i.e., the set of items the agent prefers to sell according to v'_i and the set of items the agent prefers to keep. Notice that for the purpose of v'_i, S_i, K_i , we ignore the fact that upon sale, the agent only keeps $\frac{1}{n}$ of the sale proceeds. We also

introduce the notation $\tilde{v}_i(e) = \begin{cases} \frac{1}{n} v'_i(e) & e \in S_i \\ v'_i(e) & e \in K_i. \end{cases}$

Let s_r denote the sales proceeds that each agent holds at the beginning of round r (by equal-proceeds, all agents hold the same amount). Initially, agents are not assigned any items, which we denote by $B_i = \emptyset$ for every agent i . In each round, we do the following. We define an agent i as active if it holds a value of less than $\frac{1}{n} \cdot TPS_i$ in goods assigned to it and sale proceeds s_r . I.e., an agent is active if $v_i(B_i) + s_r < \frac{1}{n} \cdot TPS_i$.

Initially, all agents are active. In each round, we ensure at least one agent becomes inactive. We do so by the following sequence of rules:

1. If there is an item e and an active agent i with $p(e) \geq TPS_i$, sell e . Update $s_r = s_r + \frac{1}{n} \cdot p(e)$.
2. Else, if there is an item e and active agent i with $v_i(e) \geq \frac{1}{n} TPS_i - s_r$, give e to agent i , i.e., set $B_i = B_i \cup \{e\}$.
3. Else, start filling a bag B with items until some agent i declares that $\tilde{v}_i(B) \geq \frac{1}{n} TPS_i - s_r$. If there is more than one such agent, then the winner of B is the agent i that generates the highest amount of sale proceeds from B . Let s_i denote the amount of these sales proceeds. B is given to i , who keeps those items that she intended to keep and sells the others. Agent i becomes inactive, and s_r, B_i are updated accordingly.

Theorem 6. *The above algorithm guarantees $\frac{1}{n} \cdot TPS_i$ to every agent i .*

Proof. Let R be the set of available goods, s be the money each agent currently holds, and T be the set of active agents. We track the algorithm's progress using the following invariant for every $k \in T$:

$$v'_k(R) + n \cdot s \geq |T| \cdot TPS_k$$

Initially, $R = \mathcal{M}$, $s = 0$, and $|T| = n$. The invariant holds with equality because $\sum_{e \in \mathcal{M}} v'_k(e) = n \cdot TPS_k$. In each round, we ensure at least one agent becomes inactive.

1. Rule 1: If $p(e) \geq TPS_i$ for some active i and $e \in R$: Sell e and update $s \leftarrow s + \frac{p(e)}{n}$. Agent i reaches the $\frac{TPS_i}{n}$ threshold and becomes inactive. For any other active k , the effective loss minus returned money is $v'_k(e) - p(e) \leq \min\{v_k(e), TPS_k\} \leq TPS_k$.
2. Rule 2: Else, if $v_i(e) \geq \frac{TPS_i}{n} - s$ for some active i and $e \in R$: Give e to i . Since Rule 1 failed, $p(e) < TPS_k$, so the effective loss to any other active k is $v'_k(e) = \max\{p(e), \min\{v_k(e), TPS_k\}\} \leq TPS_k$.
3. Rule 3 (Bag-filling): Else, begin bag-filling. As $v'_k \leq n \cdot \tilde{v}_k$, the invariant ensures $n \cdot \tilde{v}_k(R) + n \cdot s \geq |T| \cdot TPS_k$, meaning $\tilde{v}_k(R) + s \geq \frac{TPS_k}{n}$. Thus, a bag filled with all of R must trigger a claim. We can thus find a *minimal* bag $B \subseteq R$ where a set of active agents W declares $\tilde{v}_i(B) \geq \frac{TPS_i}{n} - s$. Let winner $i \in W$ be the one who maximizes the generated proceeds $d = p(B \cap S_i)$. Agent i takes B , sells $B \cap S_i$, keeps $B \cap K_i$, and becomes inactive. We update $s \leftarrow s + \frac{d}{n}$.

For any remaining active agent $k \neq i$, let $a = v'_k(B \cap K_k)$ and $b = p(B \cap S_k)$. The effective value lost from the pool is $v'_k(B) = a + b$. k 's net loss, accounting for the system-wide proceeds d returned to the pool, is $v'_k(B) - d$.

- If $b \leq d$ (winner sells at least as much as k would): The net loss is $a + b - d \leq a \leq a + \frac{b}{n} = \tilde{v}_k(B)$. By the minimality of B and the failure of prior rules, $\tilde{v}_k(B) < \frac{2TPS_k}{n} \leq TPS_k$ (as $n \geq 2$).
- If $b > d$ (winner sells less than k would): By the tie-breaking rule, $k \notin W$, so $\tilde{v}_k(B) < \frac{TPS_k}{n} - s$. Expanding this gives $a + \frac{b}{n} < \frac{TPS_k}{n} - s$. Multiplying by n yields $n \cdot a + b < TPS_k - n \cdot s$. The net loss is $a + b - d \leq a + b \leq n \cdot a + b < TPS_k - n \cdot s \leq TPS_k$.

In all cases, the net loss to any active agent k is bounded by TPS_k . Updating the invariant for the remaining items R' and new shared money s' yields:

$$v'_k(R') + n \cdot s' \geq (v'_k(R) + n \cdot s) - TPS_k \geq |T| \cdot TPS_k - TPS_k = (|T| - 1) \cdot TPS_k$$

. Since $|T|$ decreases by at least one each round, the algorithm terminates in at most n rounds, successfully guaranteeing every agent at least $\frac{t_i}{n}$.